Приложение А
Листинг программы

main.cpp:

```
 #include "Common.hpp"
#include "Globals.hpp"
#include "Helpers.hpp"
#include "Utils.hpp"
#include "ArduinoSimulatorApp.hpp"

int main() {
    setlocale(LC_ALL, "ru_RU.UTF-8");
    srand(static_cast<unsigned int>(time(nullptr)));
    Vector2u desktopSize = VideoMode::getDesktopMode().size; Vector2u
windowSize(1920, 1080);
    if (desktopSize.x < 1920 || desktopSize.y < 1080) {
        float scale = min(static_cast<float>(desktopSize.x) / 1920.f,
static_cast<float>(desktopSize.y) / 1080.f);
        windowSize.x = static_cast<unsigned int>(1920.f * scale * 0.95f);
windowSize.y = static_cast<unsigned int>(1080.f * scale * 0.95f);
    }
    RenderWindow window(VideoMode(windowSize), "Arduino Simulator",
Style::Default, State::Windowed); window.setFramerateLimit(60);
    {
        Image windowIcon;
        if (windowIcon.loadFromFile("icon.png"))
window.setIcon(windowIcon.getSize(), windowIcon.getPixelsPtr());
    }
    auto app = make_unique<ArduinoSimulatorApp>();
    auto updateViewport = [](RenderWindow& win) {
        View view(FloatRect({ 0.f, 0.f }, { 1920.f, 1080.f })); Vector2u winSize =
win.getSize();
        float windowRatio = static_cast<float>(winSize.x) /
static_cast<float>(winSize.y); float targetRatio = 16.f / 9.f;
        float sizeX = 1.f; float sizeY = 1.f; float posX = 0.f; float posY = 0.f;
        if (windowRatio > targetRatio) { sizeX = targetRatio / windowRatio; posX =
(1.f - sizeX) / 2.f; }
        else { sizeY = windowRatio / targetRatio; posY = (1.f - sizeY) / 2.f; }
        view.setViewport(FloatRect({ posX, posY }, { sizeX, sizeY }));
win.setView(view);
        };
    updateViewport(window); Clock dtClock;
    while (window.isOpen()) {
        float dt = dtClock.restart().asSeconds(); Vector2f mousePos =
window.mapPixelToCoords(Mouse::getPosition(window));
        if (app->currentState == GameState::Loading) {
            app->loadingProgress += dt * 0.4f; if (app->loadingProgress >= 1.f) { app-
>loadingProgress = 1.f; app->currentState = GameState::MainMenu; }
        }


        if (app->currentState == GameState::Simulation) {


            auto allPins = app->getAllPins();
            unordered_map<uint32_t, Pin*> pinMap;
            for (Pin* p : allPins) pinMap[p->getId()] = p;

            uint32_t servoVccId = app->servo1.getVccPin()->getId();
            uint32_t servoGndId = app->servo1.getGndPin()->getId();
            auto vccNet = app->circuitManager.getConnectedNetwork(servoVccId);
            auto gndNet = app->circuitManager.getConnectedNetwork(servoGndId);

            bool vccHas5V = false;
            bool vccHas33V = false;
            for (uint32_t id : vccNet) {
                if (pinMap.count(id)) {
                    if (pinMap[id]->getRole() == PinRole::Source5V) vccHas5V = true;
                    if (pinMap[id]->getRole() == PinRole::Source3_3V) vccHas33V =
true;
                }
            }
            bool gndHasGnd = false;
            for (uint32_t id : gndNet) {
                if (pinMap.count(id) && pinMap[id]->getRole() ==
PinRole::SourceGND) gndHasGnd = true;
            }

            uint32_t rLeft = app->resistor1.getLeftPin()->getId();
            uint32_t rRight = app->resistor1.getRightPin()->getId();
            bool resistorInServoVcc = false;
            for (uint32_t id : vccNet) {
```

Common.hpp:

```
#pragma once

#include <SFML/Graphics.hpp>
#include <SFML/Audio.hpp>
#include <SFML/Window/Clipboard.hpp>
#include <iostream>
#include <vector>
#include <memory>
#include <cmath>
#include <optional>
#include <string>
#include <functional>
#include <clocale>
#include <set>
#include <unordered_map>
#include <unordered_set>
#include <queue>
#include <algorithm>
#include <regex>
#include <random>
#include <ctime>
#include <sstream>

#ifdef _WIN32
#include <windows.h>
#endif

#include <thread>
#include <mutex>
#include <chrono>
#include <atomic>

#define SOL_ALL_SAFETIES_ON 1
#if defined(_MSC_VER)
#pragma warning(push, 0)
#endif
#include <sol/sol.hpp>
#if defined(_MSC_VER)
#pragma warning(pop)
#endif

using namespace std;
using namespace sf;
using namespace std::this_thread;
using namespace std::chrono;

enum class GameState { Loading, MainMenu, Theory, LevelSelect, Settings,
Simulation };
enum class LogicLevel { Floating, Low, High };
enum class PinRole { Normal, Source5V, Source3_3V, SourceGND, Sink };
```

```
            if (id == rLeft || id == rRight) { resistorInServoVcc = true; break; }
        }


        bool servoPowerOk = vccHas5V && !resistorInServoVcc && gndHasGnd
&& (app->servo1.getSignalPin()->getState() != LogicLevel::Floating);
        app->servo1.setPowered(servoPowerOk);
        app->servo1.update(dt);

        if (app->showErrorCod) app->btnErrorCodOk.update(mousePos);
        if (app->showErrorDiod) { app->btnErrorDiodRestart.update(mousePos);
app->btnErrorDiodMenu.update(mousePos); }
        if (app->showErrorShort) { app->btnErrorShortRestart.update(mousePos);
app->btnErrorShortMenu.update(mousePos); }
        if (app->showVictoryMenu) { app->btnVictoryMenu.update(mousePos);
app->btnVictoryRestart.update(mousePos); if (activeLevel < 5) app-
>btnVictoryNext.update(mousePos); }
        if (!app->showErrorCod && !app->showErrorDiod && !app-
>showErrorShort && !app->showVictoryMenu) app-
>btnCondition.update(mousePos);
        if (app->networkSimulator.shortCircuitDetected && !app-
>showErrorShort) {
            app->showErrorShort = true;
            app->forceStopCode();
            app->rm.playError();
        }
        if (!app->led1.isBurnedOut()) {
            uint32_t anodeId = app->led1.getAnodePin()->getId();
            uint32_t cathodeId = app->led1.getCathodePin()->getId();
            auto anodeNet = app->circuitManager.getConnectedNetwork(anodeId);
            auto cathodeNet = app-
>circuitManager.getConnectedNetwork(cathodeId);
            bool anodeHas5V = false;
            bool anodeHas33V = false;
            for (uint32_t id : anodeNet) {
                if (pinMap.count(id)) {
                    if (pinMap[id]->getRole() == PinRole::Source5V) anodeHas5V =
true;
                    if (pinMap[id]->getRole() == PinRole::Source3_3V) anodeHas33V
= true;
                }
            }
            bool cathodeHasGND = false;
            for (uint32_t id : cathodeNet) {
                if (pinMap.count(id) && pinMap[id]->getRole() ==
PinRole::SourceGND) cathodeHasGND = true;
            }
            bool resistorPresent = false;
            for (uint32_t id : anodeNet) { if (id == rLeft || id == rRight) {
resistorPresent = true; break; } }
            for (uint32_t id : cathodeNet) { if (id == rLeft || id == rRight) {
resistorPresent = true; break; } }
            if ((anodeHas5V || anodeHas33V) && cathodeHasGND) {
                if (anodeHas5V) {
                    if (resistorPresent) { app->led1.setTurnedOn(true); }
                    else { app->led1.setBurnedOut(true); app->showErrorDiod = true;
app->forceStopCode(); app->rm.playError(); }
                }
                else if (anodeHas33V) {
                    if (resistorPresent) { app->led1.setTurnedOn(false); }
                    else { app->led1.setTurnedOn(true); }
                }
            }
            else { app->led1.setTurnedOn(false); }
        }


        if (isScriptRunning && activeLevel != 5) {
            if (activeLevel <= 3 && !app->led1.isBurnedOut()) {
                if (app->led1.isTurnedOn()) {
                    if (!ledWasOn) { ledOnCount++; ledWasOn = true; }
                }
                else { ledWasOn = false; }
                if (ledOnCount >= 5 && !victoryTriggered && !levelCleared) {
                    victoryTriggered = true; victoryDelayTimer = 0.f;
                }
            }
            if ((activeLevel == 4 || activeLevel == 5) && app-
>servo1.isConnectedCorrectly()) {
```

```
                if (abs(app->servo1.getCurrentAngle() - 90.f) < 2.0f && abs(app-
>servo1.getTargetAngle() - 90.f) < 1.0f) {
                    if (!victoryTriggered && !levelCleared) { victoryTriggered = true;
victoryDelayTimer = 0.f; }
                }
            }
        }
    }
    if (app->currentState == GameState::Simulation && app-
>showStopNotification) {
        app->notificationTimer -= dt; if (app->notificationTimer <= 0.f) app-
>showStopNotification = false;
    }
    if (app->currentState == GameState::MainMenu) {
        app->btnTheory.update(mousePos); app->btnPractice.update(mousePos);
app->btnSettings.update(mousePos); app->btnHelp.update(mousePos); app-
>btnExit.update(mousePos);
    }
    else if (app->currentState == GameState::Settings) {
        app->btnToggleFS.update(mousePos); app-
>btnToggleSound.update(mousePos); app->btnSettingsBack.update(mousePos);
    }
    else if (app->currentState == GameState::Theory) {
        app->btnTheoryBack.update(mousePos); app->theorySlider.update(dt);
        if (app->theorySlider.getCurrentIdx() > 0) { if (app->btnTheoryLeft-
>getGlobalBounds().contains(mousePos)) app->btnTheoryLeft-
>setColor(Color(180, 180, 180)); else app->btnTheoryLeft-
>setColor(Color::White); }
        if (app->theorySlider.getCurrentIdx() < app->theorySlider.getTotalSlides()
- 1) { if (app->btnTheoryRight->getGlobalBounds().contains(mousePos)) app-
>btnTheoryRight->setColor(Color(180, 180, 180)); else app->btnTheoryRight-
>setColor(Color::White); }
    }
    else if (app->currentState == GameState::LevelSelect) {
        app->btnLevelBack.update(mousePos); app-
>btnLevelStart.update(mousePos); app->levelSlider.update(dt);
    }
    else if (app->currentState == GameState::Simulation) {
        if (app->showSimSettings) { app->btnSimToggleFS.update(mousePos);
app->btnSimToggleSound.update(mousePos); app-
>btnSimSettingsBack.update(mousePos); }
        else if (!app->showErrorCod && !app->showErrorDiod && !app-
>showErrorShort && !app->showVictoryMenu) {
            app->runButton.update(mousePos); app-
>stopButton.update(mousePos); app->btnSimBack.update(mousePos); app-
>btnSimSettings.update(mousePos); app->btnResetLevel.update(mousePos);
            if (app->persistentVictoryButtons) { app-
>btnSimResetTask.update(mousePos); app-
>btnPersistentToLevels.update(mousePos); if (activeLevel < 6) app-
>btnPersistentNextLevel.update(mousePos); }
        }
    }
    while (const auto event = window.pollEvent()) {
        if (event->is<Event::Closed>()) window.close();
        if (event->is<Event::Resized>()) updateViewport(window);
        if (app->currentState == GameState::Simulation) {
            if (app->showErrorCod) {
                if (const auto* mousePressed = event-
>getIf<Event::MouseButtonPressed>()) {
                    if (mousePressed->button == Mouse::Button::Left) { Vector2f
clickPos = window.mapPixelToCoords(mousePressed->position); if (app-
>btnErrorCodOk.checkClick(clickPos)) app->showErrorCod = false; }
                }
                continue;
            }
            if (app->showErrorDiod) {
                if (const auto* mousePressed = event-
>getIf<Event::MouseButtonPressed>()) {
                    if (mousePressed->button == Mouse::Button::Left) {
                        Vector2f clickPos = window.mapPixelToCoords(mousePressed-
>position);
                        if (app->btnErrorDiodRestart.checkClick(clickPos)) { if
(isScriptRunning) { stopRequested = true;
this_thread::sleep_for(chrono::milliseconds(20)); } app-
>virtualArduino.resetAllPins(); isScriptRunning = false; app-
>resetSimulationEnvironment(); }
                        else if (app->btnErrorDiodMenu.checkClick(clickPos)) {
                            if (isScriptRunning) { stopRequested = true;
this_thread::sleep_for(chrono::milliseconds(20)); } app-
```

```
>virtualArduino.resetAllPins(); isScriptRunning = false; app-
>resetSimulationEnvironment(); app->currentState = GameState::MainMenu;
                    }
                }
            }
            continue;
        }
        if (app->showErrorShort) {
            if (const auto* mousePressed = event-
>getIf<Event::MouseButtonPressed>()) {
                if (mousePressed->button == Mouse::Button::Left) {
                    Vector2f clickPos = window.mapPixelToCoords(mousePressed-
>position);
                    if (app->btnErrorShortRestart.checkClick(clickPos)) { if
(isScriptRunning) { stopRequested = true;
this_thread::sleep_for(chrono::milliseconds(20)); } app-
>virtualArduino.resetAllPins(); isScriptRunning = false; app-
>resetSimulationEnvironment(); }
                    else if (app->btnErrorShortMenu.checkClick(clickPos)) {
                        if (isScriptRunning) { stopRequested = true;
this_thread::sleep_for(chrono::milliseconds(20)); } app-
>virtualArduino.resetAllPins(); isScriptRunning = false; app-
>resetSimulationEnvironment(); app->currentState = GameState::MainMenu;
                    }
                }
            }
            continue;
        }
        if (app->showVictoryMenu) {
            if (const auto* mousePressed = event-
>getIf<Event::MouseButtonPressed>()) {
                if (mousePressed->button == Mouse::Button::Left) {
                    Vector2f clickPos = window.mapPixelToCoords(mousePressed-
>position);
                    if (app->btnVictoryMenu.checkClick(clickPos)) { app-
>showVictoryMenu = false; app->resetSimulationEnvironment(); app-
>currentState = GameState::LevelSelect; }
                    else if (app->btnVictoryRestart.checkClick(clickPos)) { app-
>showVictoryMenu = false; app->persistentVictoryButtons = true; app-
>virtualArduino.resetAllPins(); app->led1.setTurnedOn(false); }
                    else if (activeLevel < 5 && app-
>btnVictoryNext.checkClick(clickPos)) { app->showVictoryMenu = false;
activeLevel++; generateLevelTask(); app->resetSimulationEnvironment(); }
                }
            }
            continue;
        }
        if (const auto* mouseMoved = event->getIf<Event::MouseMoved>()) {
            app-
>codeTextBox.handleMouseMove(window.mapPixelToCoords(mouseMoved-
>position));
            if (app->firstPin && app->previewWire) { Vector2f
currentMousePos = window.mapPixelToCoords(mouseMoved->position); app-
>previewWire->update(app->lastPoint, currentMousePos); }
        }
        if (app->showSimSettings) {
            if (const auto* mousePressed = event-
>getIf<Event::MouseButtonPressed>()) {
                if (mousePressed->button == Mouse::Button::Left) {
                    Vector2f clickPos = window.mapPixelToCoords(mousePressed-
>position);
                    if (app->btnSimToggleFS.checkClick(clickPos)) {
                        app->isFullscreen = !app->isFullscreen;
                        app->btnToggleFS.setTexture(app->isFullscreen ? app-
>rm.btnFsOnTex : app->rm.btnFsOffTex, { 360.f, 100.f }); app-
>btnSimToggleFS.setTexture(app->isFullscreen ? app->rm.btnFsOnTex : app-
>rm.btnFsOffTex, { 360.f, 100.f });
                        if (app->isFullscreen)
window.create(VideoMode::getDesktopMode(), "Arduino Lua Simulator",
Style::Default, State::Fullscreen);
                        else window.create(VideoMode(windowSize), "Arduino Lua
Simulator", Style::Default, State::Windowed);
                        updateViewport(window);
                    }
                    if (app->btnSimToggleSound.checkClick(clickPos)) {
                        app->isSoundEnabled = !app->isSoundEnabled;
g_soundEnabled = app->isSoundEnabled;
                        app->btnToggleSound.setTexture(app->isSoundEnabled ?
app->rm.btnSoundOnTex : app->rm.btnSoundOffTex, { 360.f, 100.f }); app-
```

```
>btnSimToggleSound.setTexture(app->isSoundEnabled ? app-
>rm.btnSoundOnTex : app->rm.btnSoundOffTex, { 360.f, 100.f });
                    }
                    if (app->btnSimSettingsBack.checkClick(clickPos)) { app-
>showSimSettings = false; }
                }
            }
            continue;
        }
    }
    if (const auto* textEntered = event->getIf<Event::TextEntered>()) app-
>codeTextBox.handleTextEntered(textEntered->unicode);
    if (const auto* keyPressed = event->getIf<Event::KeyPressed>()) {
        if (keyPressed->code == Keyboard::Key::Escape) app-
>resetCurrentWire();
        app->codeTextBox.handleKeyPressed(keyPressed->code, keyPressed-
>control, keyPressed->shift);
        if (app->currentState == GameState::LevelSelect) {
            if (keyPressed->code == Keyboard::Key::Left || keyPressed->code ==
Keyboard::Key::A) app->levelSlider.moveLeft();
            if (keyPressed->code == Keyboard::Key::Right || keyPressed->code
== Keyboard::Key::D) app->levelSlider.moveRight();
        }
        else if (app->currentState == GameState::Theory) {
            if (keyPressed->code == Keyboard::Key::Left || keyPressed->code ==
Keyboard::Key::A) app->theorySlider.moveLeft();
            if (keyPressed->code == Keyboard::Key::Right || keyPressed->code
== Keyboard::Key::D) app->theorySlider.moveRight();
        }
    }
    if (const auto* mouseReleased = event-
>getIf<Event::MouseButtonReleased>()) {
        if (mouseReleased->button == Mouse::Button::Left) {
            app->codeTextBox.handleMouseRelease();
            if (app->currentState == GameState::Simulation && activeLevel >= 2
&& activeLevel != 4) { app->button1.setPressed(false, app->circuitManager, app-
>getAllPins(), app->networkSimulator); }
        }
    }
    if (const auto* mouseWheel = event-
>getIf<Event::MouseWheelScrolled>()) {
        if (app->currentState == GameState::LevelSelect) { if (mouseWheel-
>delta > 0.f) app->levelSlider.moveLeft(); else if (mouseWheel->delta < 0.f) app-
>levelSlider.moveRight(); }
        else if (app->currentState == GameState::Theory) { if (mouseWheel-
>delta > 0.f) app->theorySlider.moveLeft(); else if (mouseWheel->delta < 0.f)
app->theorySlider.moveRight(); }
        else if (app->currentState == GameState::Simulation && !app-
>showErrorCod && !app->showErrorDiod && !app->showErrorShort && !app-
>showSimSettings && !app->showVictoryMenu) {
            bool shiftPressed = Keyboard::isKeyPressed(Keyboard::Key::LShift)
|| Keyboard::isKeyPressed(Keyboard::Key::RShift); app-
>codeTextBox.handleMouseWheel(mouseWheel->delta, shiftPressed);
        }
    }
    if (const auto* mousePressed = event-
>getIf<Event::MouseButtonPressed>()) {
        Vector2f clickPos = window.mapPixelToCoords(mousePressed-
>position);
        if (app->currentState == GameState::MainMenu && mousePressed-
>button == Mouse::Button::Left) {
            if (app->btnTheory.checkClick(clickPos)) { app->currentState =
GameState::Theory; app->theorySlider.reset(); }
            if (app->btnPractice.checkClick(clickPos)) { app->currentState =
GameState::LevelSelect; app->levelSlider.reset(); }
            if (app->btnSettings.checkClick(clickPos)) app->currentState =
GameState::Settings;
            if (app->btnHelp.checkClick(clickPos)) {
#ifdef _WIN32
                ShellExecuteA(NULL, "open", "help.chm", NULL, NULL,
SW_SHOWNORMAL);
#endif
            }
            if (app->btnExit.checkClick(clickPos)) window.close();
        }
        else if (app->currentState == GameState::Settings && mousePressed-
>button == Mouse::Button::Left) {
            if (app->btnSettingsBack.checkClick(clickPos)) app->currentState =
GameState::MainMenu;
```

```
                if (app->btnToggleFS.checkClick(clickPos)) {
                    app->isFullscreen = !app->isFullscreen; app->btnToggleFS.setTexture(app->isFullscreen ? app->rm.btnFsOnTex : app->rm.btnFsOffTex, { 360.f, 100.f });
                    if (app->isFullscreen) window.create(VideoMode::getDesktopMode(), "Arduino Simulator", Style::Default, State::Fullscreen);
                    else window.create(VideoMode(windowSize), "Arduino Simulator", Style::Default, State::Windowed);
                    updateViewport(window);
                }
                if (app->btnToggleSound.checkClick(clickPos)) { app->isSoundEnabled = !app->isSoundEnabled; g_soundEnabled = app->isSoundEnabled; app->btnToggleSound.setTexture(app->isSoundEnabled ? app->rm.btnSoundOnTex : app->rm.btnSoundOffTex, { 360.f, 100.f }); }
            }
            else if (app->currentState == GameState::Theory && mousePressed->button == Mouse::Button::Left) {
                if (app->btnTheoryBack.checkClick(clickPos)) app->currentState = GameState::MainMenu;
                if (app->theorySlider.getCurrentIdx() > 0 && app->btnTheoryLeft->getGlobalBounds().contains(clickPos)) app->theorySlider.moveLeft();
                if (app->theorySlider.getCurrentIdx() < app->theorySlider.getTotalSlides() - 1 && app->btnTheoryRight->getGlobalBounds().contains(clickPos)) app->theorySlider.moveRight();
            }
            else if (app->currentState == GameState::LevelSelect && mousePressed->button == Mouse::Button::Left) {
                if (app->btnLevelBack.checkClick(clickPos)) app->currentState = GameState::MainMenu;
                int clickedLevel = app->levelSlider.checkClick(clickPos);
                if (clickedLevel != -1) { activeLevel = clickedLevel; generateLevelTask(); app->resetSimulationEnvironment(); app->currentState = GameState::Simulation; }
                else if (app->btnLevelStart.checkClick(clickPos)) { activeLevel = app->levelSlider.getCurrentIdx(); generateLevelTask(); app->resetSimulationEnvironment(); app->currentState = GameState::Simulation; }
            }
            else if (app->currentState == GameState::Simulation) {
                if (app->btnCondition.checkClick(clickPos) && mousePressed->button == Mouse::Button::Left) {
                    app->showConditionWindow = !app->showConditionWindow;
                    app->btnCondition.setTexture(app->showConditionWindow ? app->rm.btnConditionDownTex : app->rm.btnConditionUpTex, { 180.f, 90.f });
                    continue;
                }
                if (app->btnSimSettings.checkClick(clickPos) && mousePressed->button == Mouse::Button::Left) {
                    app->showSimSettings = true; app->btnSimToggleFS.setTexture(app->isFullscreen ? app->rm.btnFsOnTex : app->rm.btnFsOffTex, { 360.f, 100.f });
                    app->btnSimToggleSound.setTexture(app->isSoundEnabled ? app->rm.btnSoundOnTex : app->rm.btnSoundOffTex, { 360.f, 100.f }); continue;
                }
                if (app->persistentVictoryButtons && app->btnSimResetTask.checkClick(clickPos) && mousePressed->button == Mouse::Button::Left) {
                    if (isScriptRunning) { stopRequested = true; this_thread::sleep_for(chrono::milliseconds(20)); }
                    app->virtualArduino.resetAllPins(); isScriptRunning = false; generateLevelTask(); app->resetSimulationEnvironment(); app->persistentVictoryButtons = false; continue;
                }
                if (app->persistentVictoryButtons && mousePressed->button == Mouse::Button::Left) {
                    if (app->btnPersistentToLevels.checkClick(clickPos)) { app->resetSimulationEnvironment(); app->currentState = GameState::LevelSelect; continue; }
                    if (activeLevel < 6 && app->btnPersistentNextLevel.checkClick(clickPos)) { activeLevel++; generateLevelTask(); app->resetSimulationEnvironment(); continue; }
                }
                if (mousePressed->button == Mouse::Button::Left) {
                    if (app->contextMenu.currentState != ContextMenu::Closed) {
                        if (activeLevel == 1 || activeLevel == 3) app->contextMenu.close();
                        else app->contextMenu.handleLeftClick(clickPos, app->connections, app->circuitManager, app->getAllPins(), app->networkSimulator, [&]() { app->forceStopCode(); });
```

```
                    continue;
                }
                if (app->btnSimBack.checkClick(clickPos)) { if (isScriptRunning)
{ stopRequested = true; this_thread::sleep_for(chrono::milliseconds(20)); } app-
>virtualArduino.resetAllPins(); isScriptRunning = false; app->currentState =
GameState::MainMenu; continue; }
                if (app->codeTextBox.checkClick(clickPos)) { app-
>codeTextBox.setFocused(true); app->codeTextBox.handleMousePress(clickPos);
app->resetCurrentWire(); continue; }
                else app->codeTextBox.setFocused(false);
                if (app->runButton.checkClick(clickPos)) {
                    app->resetCurrentWire(); if (isScriptRunning) continue;
                    string code = app->codeTextBox.getText(); bool syntaxError =
false; stringstream ss(code); string line;
                    while (getline(ss, line)) {
                        size_t first = line.find_first_not_of(" \t\r\n"); if (first ==
string::npos) continue;
                        size_t last = line.find_last_not_of(" \t\r\n"); string trimmed =
line.substr(first, last - first + 1);
                        size_t commentPos = trimmed.find("//"); if (commentPos !=
string::npos) {
                            trimmed = trimmed.substr(0, commentPos); size_t lastValid
= trimmed.find_last_not_of(" \t\r\n");
                            if (lastValid != string::npos) trimmed = trimmed.substr(0,
lastValid + 1); else trimmed.clear();
                        }
                        if (trimmed.empty()) continue; if (trimmed.back() == '{' ||
trimmed.back() == '}') continue;
                        if (trimmed.rfind("void", 0) == 0 || trimmed.rfind("for", 0) ==
0 || trimmed.rfind("while", 0) == 0 || trimmed.rfind("if", 0) == 0 ||
trimmed.rfind("else", 0) == 0) continue;
                        if (trimmed.back() != ';') { syntaxError = true; break; }
                    }
                    if (syntaxError) { app->showErrorCod = true; app-
>rm.playError(); continue; }
                    isScriptRunning = true; stopRequested = false; ledOnCount = 0;
ledWasOn = false; victoryTimer = 0.f;
                    thread([&app = *app, code]() {
                        try {
                            string luaCode = code; size_t pos = 0;
                            while ((pos = luaCode.find("//", pos)) != string::npos) {
size_t endLine = luaCode.find("\n", pos); luaCode.replace(pos, 2, "--"); pos =
endLine; }
                            replaceAll(luaCode, "else if", "elseif"); regex
braceElseifRegex(R"(\}\s*elseif)"); luaCode = regex_replace(luaCode,
braceElseifRegex, "elseif");
                            regex braceElseRegex(R"(\}\s*else\b)"); luaCode =
regex_replace(luaCode, braceElseRegex, "else");
                            regex
funcRegex(R"(\b(void|int|float|double|bool|char)\s+(\w+)\s*\(([^)]*)\))"); luaCode
= regex_replace(luaCode, funcRegex, "function $2($3)");
                            regex
forRegex(R"(for\s*\(\s*int\s+(\w+)\s*=\s*(\d+)\s*;\s*\1\s*<\s*(\d+)\s*;\s*\1\s*\+\
+\s*\))"); luaCode = regex_replace(luaCode, forRegex, "for $1 = $2, $3 - 1 do ");
                            regex whileRegex(R"(while\s*\(\s*([^)]+)\s*\))"); luaCode
= regex_replace(luaCode, whileRegex, "while $1 do if stopRequested then break
end delay(5) ");
                            regex ifRegex(R"(\b(if|elseif)\s*\(((?:[^()]+|\([^()]*\))*)\))");
luaCode = regex_replace(luaCode, ifRegex, "$1 $2 then ");
                            replaceAll(luaCode, "!=", "~="); replaceAll(luaCode,
"&&", " and "); replaceAll(luaCode, "||", " or ");
                            regex notRegex(R"(!\s*([\w\(]))"); luaCode =
regex_replace(luaCode, notRegex, " not $1");
                            regex
typeClearRegex(R"(\b(int|float|double|bool|char|const)\s+)"); luaCode =
regex_replace(luaCode, typeClearRegex, "");
                            replaceAll(luaCode, "{", " "); replaceAll(luaCode, "}",
"end");
                            app.lua["setup"] = sol::lua_nil; app.lua["loop"] =
sol::lua_nil; app.lua.script(luaCode);
                            sol::function setupFunc = app.lua["setup"]; if
(setupFunc.valid()) setupFunc();
                            sol::function loopFunc = app.lua["loop"]; if
(loopFunc.valid()) { while (isScriptRunning && !stopRequested) { loopFunc();
this_thread::sleep_for(chrono::milliseconds(1)); } }
                        }
                        catch (...) { cerr << "Lua Runtime Error\n"; }
                        isScriptRunning = false;
                        }).detach();
```

```
                    continue;
                }
                if (app->stopButton.checkClick(clickPos)) { app->resetCurrentWire(); app->forceStopCode(); continue; }
                if (activeLevel >= 2 && activeLevel != 4 && app->button1.checkClick(clickPos)) { app->button1.setPressed(true, app->circuitManager, app->getAllPins(), app->networkSimulator); continue; }
                if (app->btnResetLevel.checkClick(clickPos)) {
                    app->resetCurrentWire(); if (isScriptRunning) { stopRequested = true; this_thread::sleep_for(chrono::milliseconds(20)); }
                    app->virtualArduino.resetAllPins(); isScriptRunning = false; app->forceStopCode(); app->led1.setBurnedOut(false);
                    app->showErrorDiod = false; app->showErrorCod = false; app->showErrorShort = false; app->button1.reset(); app->servo1.reset();
                    ledOnCount = 0; ledWasOn = false; victoryTriggered = false; victoryDelayTimer = 0.f;
                    if (activeLevel == 0 || activeLevel == 2 || activeLevel == 4 || activeLevel == 5) {
                        app->connections.clear();
                        {
                            lock_guard<mutex> lock(boardMutex); app->circuitManager.clearAll(); auto allPins = app->getAllPins();
                            for (Pin* pin : allPins) {
                                pin->setOccupied(false); if (pin->getArduinoPin() >= 0) pin->setRole(PinRole::Normal); else if (pin->getArduinoPin() == -2) pin->setRole(PinRole::SourceGND);
                                else if (pin->getArduinoPin() == -3) pin->setRole(PinRole::Source5V); else if (pin->getArduinoPin() == -4) pin->setRole(PinRole::Source3_3V);
                            }
                            if (activeLevel != 4) { app->circuitManager.addConnection(app->resistor1.getLeftPin()->getId(), app->resistor1.getRightPin()->getId()); }
                            app->networkSimulator.resolveLogic(allPins, app->circuitManager);
                        }
                    }
                    if (activeLevel == 1 || activeLevel == 3 || activeLevel == 5) {
                        if (activeLevel == 1) { app->codeTextBox.setText(L"void setup() {\n\n}\n\nvoid loop() {\n  \n}"); }
                        else if (activeLevel == 3) { app->codeTextBox.setText(L"void setup() {\n  \n}\n\nvoid loop() {\n  \n}"); }
                        else if (activeLevel == 5) { app->codeTextBox.setText(L"void setup() {\n}\n\nvoid loop() {\n  \n}"); }
                    }
                    continue;
                }
                bool clickedOnPin = false; vector<Pin*> allPins = app->getAllPins();
                for (Pin* pinPtr : allPins) {
                    auto& pin = *pinPtr;
                    if (pin.checkClick(clickPos)) {
                        clickedOnPin = true; if (pin.isOccupied()) break;
                        if (app->firstPin == nullptr) {
                            if (!pin.IsSelected()) {
                                app->firstPin = &pin; app->firstPin->setSelected(true); app->lastPoint = pin.getCenter(); app->previewWire = new Wire(app->lastPoint, mousePos);
                            }
                        }
                        else if (app->firstPin == &pin) { app->resetCurrentWire(); break; }
                        else if (!pin.IsSelected()) {
                            if (app->isMouseOverUI(clickPos)) continue;
                            Vector2f targetPinCenter = pin.getCenter();
                            if (isPathOutOfBounds(app->lastPoint, targetPinCenter)) break;
                            vector<Wire> temporarySegments;
                            bool ok = false;
                            for (int attempt = 0; attempt < 2 && !ok; ++attempt) {
                                temporarySegments.clear();
                                bool failed = false;
                                Vector2f mid = (attempt == 0)
                                    ? Vector2f(targetPinCenter.x, app->lastPoint.y)
                                    : Vector2f(app->lastPoint.x, targetPinCenter.y);
                                if (mid != app->lastPoint) {
                                    if (isSegmentOverlapping(app->lastPoint, mid, app->connections, app->oneWire) || app->isWireOverlappingUI(app->lastPoint, mid)) failed = true;
```

```
                                else temporarySegments.emplace_back(app->lastPoint,
mid);
                            }
                            if (!failed && targetPinCenter != mid) {
                                if (isSegmentOverlapping(mid, targetPinCenter, app-
>connections, app->oneWire) || app->isWireOverlappingUI(mid,
targetPinCenter)) failed = true;
                                else temporarySegments.emplace_back(mid,
targetPinCenter);
                            }
                            if (!failed) ok = true;
                        }
                        if (!ok) break;
                        for (const auto& seg : temporarySegments) app-
>oneWire.push_back(seg);
                        WireConnection newConn; newConn.segments = app-
>oneWire; newConn.pin1Id = app->firstPin->getId(); newConn.pin2Id =
pin.getId(); newConn.color = Color::Red; app-
>connections.push_back(newConn); app->oneWire.clear();
                        {
                            lock_guard<mutex> lock(boardMutex); app-
>circuitManager.addConnection(app->firstPin->getId(), pin.getId()); app-
>firstPin->setOccupied(true); pin.setOccupied(true);
                            app->networkSimulator.resolveLogic(app->getAllPins(),
app->circuitManager);
                        }
                        app->firstPin->setSelected(false); app->firstPin = nullptr;
delete app->previewWire; app->previewWire = nullptr; app->forceStopCode();
                    }
                    break;
                }
            }
            if (!clickedOnPin && app->firstPin != nullptr) {
                if (app->isMouseOverUI(clickPos)) continue;
                Vector2f snappedPoint = Wire::snapToGrid(app->lastPoint,
mousePos); if (app->isWireOverlappingUI(app->lastPoint, snappedPoint))
continue;
                if (hypot(snappedPoint.x - app->lastPoint.x, snappedPoint.y -
app->lastPoint.y) < 2.0f) continue;
                if (isPathOutOfBounds(app->lastPoint, snappedPoint) ||
isSegmentOverlapping(app->lastPoint, snappedPoint, app->connections, app-
>oneWire)) continue;
                app->oneWire.emplace_back(app->lastPoint, snappedPoint);
app->lastPoint = snappedPoint; delete app->previewWire; app->previewWire =
new Wire(app->lastPoint, mousePos);
            }
        }
        if (mousePressed->button == Mouse::Button::Right) {
            if (activeLevel == 1 || activeLevel == 3) continue;
            bool wireFound = false;
            for (size_t i = 0; i < app->connections.size(); ++i) {
                for (const auto& seg : app->connections[i].segments) {
                    if (getDistanceToSegment(clickPos, seg.getStart(),
seg.getEnd()) < 6.f) {
                        app->contextMenu.open(clickPos, static_cast<int>(i));
wireFound = true; break;
                    }
                }
                if (wireFound) break;
            }
            if (!wireFound) app->contextMenu.close(); continue;
        }
    }
  }
  if (app->currentState == GameState::Simulation && victoryTriggered &&
!app->showVictoryMenu && !levelCleared && activeLevel != 5) {
      victoryDelayTimer += dt;
      if (victoryDelayTimer >= 1.25f) {
          app->showVictoryMenu = true; levelCleared = true; victoryTriggered =
false; stopRequested = true; isScriptRunning = false;
          app->virtualArduino.resetAllPins(); app->led1.setTurnedOn(false);
          app->rm.playVictory();
      }
  }
  window.clear();
  if (app->currentState == GameState::Loading) {
```

```
        window.clear(Color(40, 40, 40)); Text loadTitle = app->titleText;
loadTitle.setPosition({ 960.f, 400.f }); window.draw(loadTitle);
window.draw(app->loadingBarBg);
        app->loadingBarFill.setSize({ 600.f * app->loadingProgress, 40.f });
window.draw(app->loadingBarFill);
        if (static_cast<int>(app->loadingProgress * 5) % 2 == 0) {
            Text blinkText(app->rm.font, L"Загрузка . . .", 32);
blinkText.setFillColor(Color::White);
            FloatRect bounds = blinkText.getLocalBounds(); blinkText.setOrigin({
bounds.size.x / 2.f, bounds.size.y / 2.f }); blinkText.setPosition({ 960.f, 650.f });
window.draw(blinkText);
        }
    }
    else if (app->currentState == GameState::MainMenu) {
        window.draw(app->menuBgSprite); window.draw(app->titleText); app-
>btnTheory.draw(window); app->btnPractice.draw(window); app-
>btnSettings.draw(window); app->btnExit.draw(window); app-
>btnHelp.draw(window);
    }
    else if (app->currentState == GameState::Settings) { window.draw(app-
>levelBgSprite); app->btnToggleFS.draw(window); app-
>btnToggleSound.draw(window); app->btnSettingsBack.draw(window); }
    else if (app->currentState == GameState::Theory) {
        window.draw(app->levelBgSprite); app->theorySlider.draw(window);
        if (app->theorySlider.getCurrentIdx() > 0) window.draw(*app-
>btnTheoryLeft); if (app->theorySlider.getCurrentIdx() < app-
>theorySlider.getTotalSlides() - 1) window.draw(*app->btnTheoryRight);
        app->btnTheoryBack.draw(window);
    }
    else if (app->currentState == GameState::LevelSelect) { window.draw(app-
>levelBgSprite); app->levelSlider.draw(window); app-
>btnLevelBack.draw(window); app->btnLevelStart.draw(window); }
    else if (app->currentState == GameState::Simulation) {
        window.draw(app->levelBgSprite); if (app->boardSprite)
window.draw(*app->boardSprite); else window.draw(app->fallback);
        for (auto& pin : app->boardPins) pin.draw(window);
        if (activeLevel != 4) { app->resistor1.draw(window); app-
>led1.draw(window); }
        if (activeLevel >= 2 && activeLevel != 4) app->button1.draw(window);
        if (activeLevel == 4 || activeLevel == 5) app->servo1.draw(window);
        for (const auto& conn : app->connections) conn.draw(window);
        for (const auto& wire : app->oneWire) wire.draw(window);
        if (app->previewWire) app->previewWire->draw(window);
        app->codeTextBox.draw(window); app->runButton.draw(window); app-
>stopButton.draw(window); app->btnResetLevel.draw(window);
        app->btnCondition.draw(window); app->btnSimBack.draw(window); app-
>btnSimSettings.draw(window);
        if (app->persistentVictoryButtons) { app-
>btnSimResetTask.draw(window); app->btnPersistentToLevels.draw(window); if
(activeLevel < 6) app->btnPersistentNextLevel.draw(window); }
        app->contextMenu.draw(window);
        if (app->showSimSettings) { RectangleShape overlay({ 1920.f, 1080.f });
overlay.setFillColor(Color(0, 0, 0, 150)); window.draw(overlay); app-
>btnSimToggleFS.draw(window); app->btnSimToggleSound.draw(window); app-
>btnSimSettingsBack.draw(window); }
        if (app->showConditionWindow) { window.draw(app->conditionRect);
window.draw(app->conditionText); }

        if (app->showStopNotification) { window.draw(app->notificationRect);
window.draw(app->notificationText); }

        if (app->showErrorCod) { RectangleShape overlay({ 1920.f, 1080.f });
overlay.setFillColor(Color(0, 0, 0, 200)); window.draw(overlay);
window.draw(app->errorCodSprite); app->btnErrorCodOk.draw(window); }
        if (app->showErrorDiod) { RectangleShape overlay({ 1920.f, 1080.f });
overlay.setFillColor(Color(0, 0, 0, 200)); window.draw(overlay);
window.draw(app->errorDiodSprite); app->btnErrorDiodRestart.draw(window);
app->btnErrorDiodMenu.draw(window); }
        if (app->showErrorShort) { RectangleShape overlay({ 1920.f, 1080.f });
overlay.setFillColor(Color(0, 0, 0, 200)); window.draw(overlay);
window.draw(app->errorShortSprite); app->btnErrorShortRestart.draw(window);
app->btnErrorShortMenu.draw(window); }
        if (app->showVictoryMenu && activeLevel != 5) {
            RectangleShape overlay({ 1920.f, 1080.f });
overlay.setFillColor(Color(0, 0, 0, 150)); window.draw(overlay);
            window.draw(app->victoryBgSprite); app-
>btnVictoryMenu.draw(window); app->btnVictoryRestart.draw(window);
            if (activeLevel < 5) app->btnVictoryNext.draw(window);
        }
```

```
        }
        window.display();
    }
    if (isScriptRunning) { stopRequested = true;
this_thread::sleep_for(chrono::milliseconds(30)); }
    return 0;
}
```

ArduinoSimulatorApp.cpp:

```
#include "ArduinoSimulatorApp.hpp"

vector<FloatRect> ArduinoSimulatorApp::getUIBounds()
{
    vector<FloatRect> bounds;
    bounds.push_back(btnSimSettings.getGlobalBounds());
bounds.push_back(btnSimBack.getGlobalBounds());
bounds.push_back(codeTextBox.getGlobalBounds());
    bounds.push_back(runButton.getGlobalBounds());
bounds.push_back(stopButton.getGlobalBounds());
bounds.push_back(btnResetLevel.getGlobalBounds());
    bounds.push_back(btnCondition.getGlobalBounds());
    if (showConditionWindow)
bounds.push_back(conditionRect.getGlobalBounds());
    if (persistentVictoryButtons) {
        bounds.push_back(btnSimResetTask.getGlobalBounds());
bounds.push_back(btnPersistentToLevels.getGlobalBounds());
        if (activeLevel < 6)
bounds.push_back(btnPersistentNextLevel.getGlobalBounds());
    }
    return bounds;
}

bool ArduinoSimulatorApp::isWireOverlappingUI(Vector2f s, Vector2f e)
{
    float minX = min(s.x, e.x) - 4.f; float maxX = max(s.x, e.x) + 4.f;
    float minY = min(s.y, e.y) - 4.f; float maxY = max(s.y, e.y) + 4.f;
    FloatRect lineAABB({ minX, minY }, { maxX - minX, maxY - minY });
    for (const auto& rect : getUIBounds()) { if
(lineAABB.findIntersection(rect).has_value()) return true; }
    return false;
}

bool ArduinoSimulatorApp::isMouseOverUI(Vector2f pos)
{
    for (const auto& rect : getUIBounds()) { if (rect.contains(pos)) return true; }
return false;
}

vector<Pin*> ArduinoSimulatorApp::getAllPins()
{
    vector<Pin*> allPins;
    for (auto& pin : boardPins) allPins.push_back(&pin);
    if (activeLevel != 4) { allPins.push_back(resistor1.getLeftPin());
allPins.push_back(resistor1.getRightPin());
allPins.push_back(led1.getAnodePin()); allPins.push_back(led1.getCathodePin());
}
    if (activeLevel >= 2 && activeLevel != 4) {
allPins.push_back(button1.getLeftPin());
allPins.push_back(button1.getRightPin()); }
    if (activeLevel == 4 || activeLevel == 5) {
allPins.push_back(servo1.getGndPin()); allPins.push_back(servo1.getVccPin());
allPins.push_back(servo1.getSignalPin()); }
    return allPins;
}

void ArduinoSimulatorApp::forceStopCode()
{
    if (isScriptRunning) {
        stopRequested = true;
        this_thread::sleep_for(chrono::milliseconds(20));
        virtualArduino.resetAllPins();
        isScriptRunning = false;
        notificationText.setString(L"Код остановлен из-за изменений");
        FloatRect ntBounds = notificationText.getLocalBounds();
        notificationText.setOrigin({ ntBounds.size.x / 2.f, ntBounds.size.y / 2.f });
        notificationText.setPosition({ 1075.f, 280.f });
        showStopNotification = true;
        notificationTimer = 3.f;
        rm.playStop();
    }
```

ArduinoSimulatorApp.hpp:

```
#pragma once

#include "Common.hpp"
#include "Globals.hpp"
#include "ResourceManager.hpp"
#include "Pin.hpp"
#include "CircuitManager.hpp"
#include "ComponentNetwork.hpp"
#include "VirtualArduino.hpp"
#include "Buttons.hpp"
#include "LevelCarousel.hpp"
#include "TheoryCarousel.hpp"
#include "TextBox.hpp"
#include "Wire.hpp"
#include "ContextMenu.hpp"
#include "Components.hpp"
#include "Utils.hpp"

struct ArduinoSimulatorApp {

    ResourceManager rm;
    GameState currentState = GameState::Loading;
    float loadingProgress = 0.f;
    RectangleShape loadingBarBg; RectangleShape loadingBarFill;
    Button btnTheory; Button btnPractice; Button btnSettings; Button btnExit;
Button btnHelp; Button btnTheoryBack;
    unique_ptr<Sprite> btnTheoryLeft; unique_ptr<Sprite> btnTheoryRight;
    bool isFullscreen = false; bool isSoundEnabled = true;
    Button btnToggleFS; Button btnToggleSound; Button btnSettingsBack;
    bool showSimSettings = false;
    Button btnSimToggleFS; Button btnSimToggleSound; Button
btnSimSettingsBack;
    LevelCarousel levelSlider; TheoryCarousel theorySlider;
    Button btnLevelBack; Button btnLevelStart;
    Sprite levelBgSprite; Text titleText;
    CircuitManager circuitManager; ComponentNetwork networkSimulator;
ContextMenu contextMenu; TextBox codeTextBox;
    Button runButton; Button stopButton; Button btnResetLevel; Button
btnSimBack; Button btnSimSettings; Button btnCondition; Button
btnSimResetTask;
    bool showVictoryMenu = false;
    Button btnPersistentToLevels; Button btnPersistentNextLevel;
    bool persistentVictoryButtons = false;
    bool showErrorCod = false; bool showErrorDiod = false; bool
showErrorShort = false; bool showConditionWindow = false;
    Sprite errorCodSprite; Sprite errorDiodSprite; Sprite errorShortSprite;
    TransparentButton btnErrorCodOk; TransparentButton
btnErrorDiodRestart; TransparentButton btnErrorDiodMenu;
TransparentButton btnErrorShortRestart; TransparentButton
btnErrorShortMenu;
    Sprite victoryBgSprite;
    TransparentButton btnVictoryMenu; TransparentButton btnVictoryRestart;
TransparentButton btnVictoryNext;
    RectangleShape conditionRect; Text conditionText;
    optional<Sprite> boardSprite; RectangleShape fallback;
    vector<Pin> boardPins;
    Resistor resistor1; LED led1; ArduinoButton button1; ServoComponent
servo1;
    sol::state lua; VirtualArduino virtualArduino;
    vector<WireConnection> connections; vector<Wire> oneWire;
    Pin* firstPin = nullptr; Vector2f lastPoint; Wire* previewWire = nullptr;
    bool showStopNotification = false; float notificationTimer = 0.f;
    Sprite menuBgSprite;
    RectangleShape notificationRect; Text notificationText;   vector<FloatRect>
getUIBounds();
    bool isWireOverlappingUI(Vector2f s, Vector2f e);
    bool isMouseOverUI(Vector2f pos);
    vector<Pin*> getAllPins();
    void forceStopCode();
    ArduinoSimulatorApp();
    void resetCurrentWire();
```

```
}


ArduinoSimulatorApp::ArduinoSimulatorApp()
    : btnTheory(
        { 780.f, 320.f }, { 360.f, 100.f }, rm.btnTheoryTex),
    btnPractice({ 780.f, 440.f }, { 360.f, 100.f }, rm.btnPracticeTex),
    btnSettings({ 780.f, 560.f }, { 360.f, 100.f }, rm.btnSettingsTex),
    btnExit({ 780.f, 680.f }, { 360.f, 100.f }, rm.btnExitTex),
    btnHelp({ 1680.f, 985.f }, { 200.f, 90.f }, rm.btnHelpTex),
    btnTheoryBack({ 50.f, 985.f }, { 200.f, 90.f }, rm.btnBackTex),
    btnToggleFS({ 780.f, 380.f }, { 360.f, 100.f }, rm.btnFsOffTex),
    btnToggleSound({ 780.f, 500.f }, { 360.f, 100.f }, rm.btnSoundOnTex),
    btnSettingsBack({ 780.f, 620.f }, { 360.f, 100.f }, rm.btnBackTex),
    btnSimToggleFS({ 780.f, 400.f }, { 360.f, 100.f }, rm.btnFsOffTex),
    btnSimToggleSound({ 780.f, 520.f }, { 360.f, 100.f }, rm.btnSoundOnTex),
    btnSimSettingsBack({ 780.f, 640.f }, { 360.f, 100.f }, rm.btnBackTex),
    levelSlider(rm.font, rm),
    theorySlider(rm),
    btnLevelBack({ 50.f, 985.f }, { 200.f, 90.f }, rm.btnBackTex),
    btnLevelStart({ 780.f, 870.f }, { 360.f, 100.f }, rm.btnStartTex),
    levelBgSprite(rm.levelBgTex),
    menuBgSprite(rm.menuBgTex),
    titleText(rm.font, L"ARDUINO SIMULATOR", 64),
    contextMenu(rm.font),
    codeTextBox({ 50.f, 50.f }, { 580.f, 820.f }, rm.font),
    runButton({ 50.f, 885.f }, { 180.f, 90.f }, rm.btnRunTex),
    stopButton({ 250.f, 885.f }, { 180.f, 90.f }, rm.btnStopTex),
    btnResetLevel({ 450.f, 885.f }, { 180.f, 90.f }, rm.btnResetTex),
    btnSimBack({ 50.f, 985.f }, { 300.f, 90.f }, rm.btnMenuGoTex),
    btnSimSettings({ 1720.f, 20.f }, { 180.f, 90.f }, rm.btnSimSettingsTex),
    btnCondition({ 1520.f, 20.f }, { 180.f, 90.f }, rm.btnConditionUpTex),
    btnSimResetTask({ 1590.f, 120.f }, { 310.f, 90.f }, rm.btnSimResetTaskTex),
    btnPersistentToLevels({ 1450.f, 985.f }, { 200.f, 90.f }, rm.btnToLevelsTex),
    btnPersistentNextLevel({ 1670.f, 985.f }, { 220.f, 90.f }, rm.btnNextLevelTex),
    errorCodSprite(rm.errorCodTex),
    errorDiodSprite(rm.errorDiodTex),
    errorShortSprite(rm.errorShortTex),
    btnErrorCodOk(960.f, 675.f, 210.f, 65.f),
    btnErrorDiodRestart(820.f, 677.5f, 200.f, 82.5f),
    btnErrorDiodMenu(1095.f, 677.5f, 200.f, 82.5f),
    btnErrorShortRestart(820.f, 677.5f, 200.f, 82.5f),
    btnErrorShortMenu(1105.f, 677.5f, 210.f, 82.5f),
    victoryBgSprite(rm.victoryBgTex),
    btnVictoryMenu(965.f, 680.f, 135.f, 90.f),
    btnVictoryRestart(780.f, 680.f, 135.f, 90.f),
    btnVictoryNext(1140.f, 680.f, 135.f, 90.f),
    fallback({ 690.f, 530.f }),
    resistor1(Vector2f(750.f, 650.f), rm),
    led1(Vector2f(990.f, 590.f), rm),
    button1(Vector2f(1250.f, 620.f), rm),
    servo1(Vector2f(1000.f, 750.f), rm),
    virtualArduino([this]() { return this->getAllPins(); }, circuitManager,
networkSimulator),
    notificationText(rm.font, L"Код остановлен из-за изменений", 22),
    conditionText(rm.font)
{
    titleText.setFillColor(Color::White); FloatRect titleBounds =
titleText.getLocalBounds();
    titleText.setOrigin({ titleBounds.size.x / 2.f, titleBounds.size.y / 2.f });
titleText.setPosition({ 960.f, 200.f });
    loadingBarBg.setSize({ 600.f, 40.f }); loadingBarBg.setOrigin({ 300.f, 20.f });
loadingBarBg.setPosition({ 960.f, 540.f });
    loadingBarBg.setFillColor(Color::Transparent);
loadingBarBg.setOutlineThickness(2.f);
loadingBarBg.setOutlineColor(Color::White);
    loadingBarFill.setSize({ 0.f, 40.f }); loadingBarFill.setOrigin({ 0.f, 20.f });
loadingBarFill.setPosition({ 660.f, 540.f });
loadingBarFill.setFillColor(Color::White);
    conditionRect.setSize({ 820.f, 230.f }); conditionRect.setFillColor(Color(40, 40,
40, 240));
    conditionRect.setOutlineThickness(3.f);
conditionRect.setOutlineColor(Color::Cyan);
    conditionRect.setOrigin({ 410.f, 115.f }); conditionRect.setPosition({ 1075.f,
280.f });
    conditionText.setCharacterSize(24); conditionText.setFillColor(Color::White);
    notificationRect.setSize({ 820.f, 230.f });
    notificationRect.setFillColor(Color(200, 50, 50, 240));
    notificationRect.setOutlineThickness(3.f);
    void createAutoWire(Pin* p1, Pin* p2, Color color);
    void createRoutedWire(Pin* boardPin, Pin* compPin, Color color, float
offset);
    void resetSimulationEnvironment();
};
```

```
   notificationRect.setOutlineColor(Color::White);
   notificationRect.setOrigin({ 410.f, 115.f });
   notificationRect.setPosition({ 1075.f, 280.f });
   notificationText.setFillColor(Color::White);
   btnTheoryLeft = make_unique<Sprite>(rm.arrowLeftTex); btnTheoryRight =
make_unique<Sprite>(rm.arrowRightTex);
   FloatRect arrLBounds = btnTheoryLeft->getLocalBounds(); btnTheoryLeft-
>setOrigin({ arrLBounds.size.x / 2.f, arrLBounds.size.y / 2.f });
   btnTheoryLeft->setScale({ 80.f / arrLBounds.size.x, 80.f / arrLBounds.size.y });
btnTheoryLeft->setPosition({ 260.f, 500.f });
   FloatRect arrRBounds = btnTheoryRight->getLocalBounds(); btnTheoryRight-
>setOrigin({ arrRBounds.size.x / 2.f, arrRBounds.size.y / 2.f });
   btnTheoryRight->setScale({ 80.f / arrRBounds.size.x, 80.f / arrRBounds.size.y
}); btnTheoryRight->setPosition({ 1660.f, 500.f });
   if (rm.errorCodTex.getSize().x > 0) {
      Vector2f sz = errorCodSprite.getLocalBounds().size;
errorCodSprite.setOrigin({ sz.x / 2.f, sz.y / 2.f });
      errorCodSprite.setPosition({ 960.f, 540.f });
   }
   if (rm.errorDiodTex.getSize().x > 0) {
      Vector2f sz = errorDiodSprite.getLocalBounds().size;
errorDiodSprite.setOrigin({ sz.x / 2.f, sz.y / 2.f });
      errorDiodSprite.setPosition({ 960.f, 540.f });
   }
   if (rm.errorShortTex.getSize().x > 0) {
      Vector2f sz = errorShortSprite.getLocalBounds().size;
errorShortSprite.setOrigin({ sz.x / 2.f, sz.y / 2.f });
      errorShortSprite.setPosition({ 960.f, 540.f });
   }
   if (rm.victoryBgTex.getSize().x > 0) {
      Vector2f sz = victoryBgSprite.getLocalBounds().size;
victoryBgSprite.setOrigin({ sz.x / 2.f, sz.y / 2.f });
      victoryBgSprite.setPosition({ 960.f, 540.f });
   }
   if (rm.hasBoardTexture) {
      boardSprite.emplace(rm.boardTex); Vector2f size = boardSprite-
>getLocalBounds().size;
      boardSprite->setScale({ 500.f / size.x, 287.5f / size.y }); boardSprite-
>setPosition({ 1380.f, 567.5f }); add_pins(boardPins, rm, *boardSprite);
   }
   else { fallback.setPosition({ 1000.f, 375.f }); fallback.setFillColor(Color::Blue);
}
   Vector2u bgSize = rm.levelBgTex.getSize(); if (bgSize.x > 0 && bgSize.y > 0) {
      levelBgSprite.setScale({ 1920.f / bgSize.x, 1080.f / bgSize.y });
   }
   Vector2u menuBgSize = rm.menuBgTex.getSize(); if (menuBgSize.x > 0 &&
menuBgSize.y > 0) {
      menuBgSprite.setScale({ 1920.f / menuBgSize.x, 1080.f / menuBgSize.y });
   }
   led1.init();
   codeTextBox.setOnChangeCallback([this]() { this->forceStopCode(); });
   lua.open_libraries(sol::lib::base, sol::lib::math);
   lua["A0"] = 14; lua["A1"] = 15; lua["A2"] = 16; lua["A3"] = 17; lua["A4"] = 18;
lua["A5"] = 19;
   lua["GND"] = -2; lua["V5"] = -3; lua["V33"] = -4; lua["HIGH"] = 1; lua["LOW"]
= 0;
   lua["high"] = 1; lua["low"] = 0; lua["OUTPUT"] = 1; lua["INPUT"] = 0;
   lua["pinMode"] = [&](int pinNum, int mode) {
virtualArduino.pinMode(pinNum, mode); };
   lua["delay"] = [&](int ms) {
      auto start = chrono::steady_clock::now();
      while (chrono::steady_clock::now() - start < chrono::milliseconds(ms)) {
         if (stopRequested) throw sol::error("Скрипт принудительно
остановлен");
         this_thread::sleep_for(chrono::milliseconds(5));
      }
      };
   lua["digitalWrite"] = [&](int pinNum, int value) {
virtualArduino.digitalWrite(pinNum, value); };
   lua["digitalRead"] = [&](int pinNum) -> int { return
virtualArduino.digitalRead(pinNum); };
   lua["board"] = &virtualArduino;
   lua["servo"] = lua.create_table();

   lua["servo"]["write"] = [this](float angle) {
      float clampedAngle = clamp(angle, 0.f, 180.f);
      this->servo1.setTargetAngle(clampedAngle);
      while (this->servo1.isConnectedCorrectly() && abs(this-
>servo1.getCurrentAngle() - clampedAngle) > 0.5f) {
```

```
        if (stopRequested) break;
        this_thread::sleep_for(chrono::milliseconds(10));
      }
      };
  lua["servo"]["read"] = [this]() -> float { return this->servo1.getCurrentAngle();
};
}


void ArduinoSimulatorApp::resetCurrentWire()
{
  if (firstPin) { firstPin->setSelected(false); firstPin = nullptr; } delete
previewWire; previewWire = nullptr; oneWire.clear();
}

void ArduinoSimulatorApp::createAutoWire(Pin* p1, Pin* p2, Color color)
{
  Vector2f s = p1->getCenter(); Vector2f e = p2->getCenter();
  WireConnection newConn; Vector2f intermediate(e.x, s.y);
  if (intermediate != s) newConn.segments.emplace_back(s, intermediate);
  if (e != intermediate) newConn.segments.emplace_back(intermediate, e);
  newConn.pin1Id = p1->getId(); newConn.pin2Id = p2->getId(); newConn.color
= color;
  for (auto& w : newConn.segments) w.setColor(color);
  connections.push_back(newConn);          p1->setOccupied(true);          p2-
>setOccupied(true);
  circuitManager.addConnection(p1->getId(), p2->getId());
}

void ArduinoSimulatorApp::createRoutedWire(Pin* boardPin, Pin* compPin,
Color color, float offset)
{
  Vector2f s = boardPin->getCenter(); Vector2f e = compPin->getCenter();
WireConnection newConn;
  float bypassY = (s.y < 500.f) ? (140.f + offset) : (910.f - offset);
  float bypassX = 470.f;
  if (e.x > 900.f) bypassX = 1120.f; else if (e.x > 650.f) bypassX = 920.f; else
bypassX = 470.f;
  if (compPin == button1.getLeftPin()) bypassX = e.x - offset;
  else if (compPin == button1.getRightPin()) bypassX = e.x + offset;
  newConn.segments.emplace_back(e, Vector2f(bypassX, e.y));
  newConn.segments.emplace_back(Vector2f(bypassX, e.y), Vector2f(bypassX,
bypassY));
  newConn.segments.emplace_back(Vector2f(bypassX, bypassY), Vector2f(s.x,
bypassY));
  newConn.segments.emplace_back(Vector2f(s.x, bypassY), s);
  newConn.pin1Id = boardPin->getId(); newConn.pin2Id = compPin->getId();
newConn.color = color;
  for (auto& w : newConn.segments) w.setColor(color);
  connections.push_back(newConn); boardPin->setOccupied(true); compPin-
>setOccupied(true);
  circuitManager.addConnection(boardPin->getId(), compPin->getId());
}


void ArduinoSimulatorApp::resetSimulationEnvironment()
{
  resetCurrentWire(); connections.clear(); circuitManager.clearAll();
  led1.setBurnedOut(false); showErrorDiod = false; showVictoryMenu = false;
showErrorCod = false;
  showErrorShort = false; networkSimulator.shortCircuitDetected = false;

  if (activeLevel == 5) {
    showConditionWindow = false;
  }
  else {
    showConditionWindow = true;
    btnCondition.setTexture(rm.btnConditionDownTex, { 180.f, 90.f });
  }

  button1.reset(); servo1.reset(); persistentVictoryButtons = false;
  ledOnCount = 0; ledWasOn = false; victoryTimer = 0.f; victoryTriggered = false;
victoryDelayTimer = 0.f; levelCleared = false;
  auto allPins = getAllPins();
  for (Pin* pin : allPins) {
    pin->setOccupied(false);
    if (pin->getArduinoPin() >= 0) pin->setRole(PinRole::Normal);
    else if (pin->getArduinoPin() == -2) pin->setRole(PinRole::SourceGND);
    else if (pin->getArduinoPin() == -3) pin->setRole(PinRole::Source5V);
```

```
        else if (pin->getArduinoPin() == -4) pin->setRole(PinRole::Source3_3V);
    }
    networkSimulator.resLeftId = resistor1.getLeftPin()->getId();
    networkSimulator.resRightId = resistor1.getRightPin()->getId();
    networkSimulator.resolveLogic(allPins, circuitManager);
    if (activeLevel == 0 || activeLevel == 2) {
        conditionText.setString(L"УСЛОВИЕ ЗАДАЧИ:\n\nСоедини схему для данного кода, чтобы\nсветодиод засветился минимум пять раз\nза один запуск программы.");
    }
    else if (activeLevel == 1 || activeLevel == 3) {
        conditionText.setString(L"УСЛОВИЕ ЗАДАЧИ:\n\nНапиши код, чтобы светодиод засветился\nминимум пять раз за один запуск программы.");
    }
    else if (activeLevel == 4) {
        conditionText.setString(L"УСЛОВИЕ ЗАДАЧИ:\n\nСоедини схему для данного кода, чтобы\nсервопривод выполнил поворот\nшлагбаума на 90 градусов.");
    }
    else {
        conditionText.setString(L"Твои возможности безграничны,\na пространство для экспериментов открыто!");
    }
    FloatRect ctBounds = conditionText.getLocalBounds();
    conditionText.setOrigin({ ctBounds.size.x / 2.f, ctBounds.size.y / 2.f });
    conditionText.setPosition({ 1075.f, 280.f });
    if (activeLevel != 4) { circuitManager.addConnection(resistor1.getLeftPin()->getId(), resistor1.getRightPin()->getId()); }
    if (activeLevel == 0) {
        string highLabel = getPinLabel(genPinHigh); string lowLabel = getPinLabel(genPinLow);
        wstring code = L"void setup() {\n  pinMode(" + wstring(highLabel.begin(), highLabel.end()) + L", OUTPUT);\n  pinMode(" + wstring(lowLabel.begin(), lowLabel.end()) + L", OUTPUT);\n}\n\nvoid loop() {\n  for (int i = 0; i < 5; i++) {\n    digitalWrite(" + wstring(highLabel.begin(), highLabel.end()) + L", HIGH);\n    digitalWrite(" + wstring(lowLabel.begin(), lowLabel.end()) + L", LOW);\n    delay(500);\n    digitalWrite(" + wstring(highLabel.begin(), highLabel.end()) + L", LOW);\n    delay(500);\n  }\n}";
        codeTextBox.setText(code); codeTextBox.setEditable(false);
    }
    else if (activeLevel == 1) {
        codeTextBox.setText(L"void setup() {\n\n}\n\nvoid loop() {\n  \n}"); codeTextBox.setEditable(true);
        Pin* pHigh = nullptr; Pin* pLow = nullptr;
        for (auto& pin : boardPins) {
            if (pin.getArduinoPin() == genPinHigh) pHigh = &pin; if (pin.getArduinoPin() == genPinLow) pLow = &pin;
        }
        if (pHigh && pLow) {
            createRoutedWire(pHigh, resistor1.getLeftPin(), Color::Red, 0.f);
            Vector2f s2 = resistor1.getRightPin()->getCenter(); Vector2f e2 = led1.getAnodePin()->getCenter(); WireConnection conn2;
            Vector2f intermediate2(e2.x, s2.y); if (intermediate2 != s2) conn2.segments.emplace_back(s2, intermediate2); if (e2 != intermediate2) conn2.segments.emplace_back(intermediate2, e2); conn2.pin1Id = resistor1.getRightPin()->getId(); conn2.pin2Id = led1.getAnodePin()->getId(); conn2.color = Color::Green;
            for (auto& w : conn2.segments) w.setColor(Color::Green); connections.push_back(conn2);
            resistor1.getRightPin()->setOccupied(true); led1.getAnodePin()->setOccupied(true); circuitManager.addConnection(resistor1.getRightPin()->getId(), led1.getAnodePin()->getId());
            createRoutedWire(pLow, led1.getCathodePin(), Color::Blue, 25.f);
        }
        networkSimulator.resolveLogic(getAllPins(), circuitManager);
    }
    else if (activeLevel == 2) {
        codeTextBox.setEditable(false);
        string highLabel = getPinLabel(genPinHigh); string lowLabel = getPinLabel(genPinLow);
        if (btnAsSwitchLevel3) {
            wstring code = L"void setup() {\n  pinMode(" + wstring(highLabel.begin(), highLabel.end()) + L", OUTPUT);\n}\n\nvoid loop() {\n  digitalWrite(" + wstring(highLabel.begin(), highLabel.end()) + L", HIGH);\n}";
            codeTextBox.setText(code);
        }
        else {
            wstring code = L"void setup() {\n  pinMode(" + wstring(highLabel.begin(), highLabel.end()) + L", OUTPUT);\n  pinMode(" + wstring(lowLabel.begin(),
```

```
lowLabel.end()) + L", INPUT);\n}\n\nvoid loop() {\n  if (digitalRead(" + wstring(lowLabel.begin(), lowLabel.end()) + L") == HIGH) {\n  for (int i = 0; i < 5; i++) {\n  digitalWrite(" + wstring(highLabel.begin(), highLabel.end()) + L", HIGH);\n  digitalWrite(" + wstring(lowLabel.begin(), lowLabel.end()) + L", LOW);\n  delay(500);\n  digitalWrite(" + wstring(highLabel.begin(), highLabel.end()) + L", LOW);\n  delay(500);\n  }\n  }\n}";
        codeTextBox.setText(code);
      }
    }
    else if (activeLevel == 3) {
      codeTextBox.setEditable(true);
      Pin* pHigh = nullptr; Pin* pLow = nullptr; Pin* pGnd = nullptr; Pin* p5V = nullptr;
      for (auto& pin : boardPins) {
        if (pin.getArduinoPin() == genPinHigh) pHigh = &pin; if (pin.getArduinoPin() == genPinLow) pLow = &pin;
        if (pin.getArduinoPin() == -2) pGnd = &pin; if (pin.getArduinoPin() == -3) p5V = &pin;
      }
      if (btnAsSwitchLevel4) {
        codeTextBox.setText(L"void setup() {\n  \n}\n\nvoid loop() {\n  \n}");
        if (pHigh && pGnd) {
          createRoutedWire(pHigh, button1.getLeftPin(), Color::Red, 10.f);
          Pin* p1 = button1.getRightPin(); Pin* p2 = resistor1.getLeftPin();
          Vector2f s = p1->getCenter(); Vector2f e = p2->getCenter();
          WireConnection newConn; float bypassY = 560.f;
          newConn.segments.emplace_back(s, Vector2f(s.x, bypassY));
          newConn.segments.emplace_back(Vector2f(s.x, bypassY), Vector2f(e.x, bypassY));
          newConn.segments.emplace_back(Vector2f(e.x, bypassY), e);
          newConn.pin1Id = p1->getId(); newConn.pin2Id = p2->getId(); newConn.color = Color::Yellow;
          for (auto& w : newConn.segments) w.setColor(Color::Yellow); connections.push_back(newConn);
          p1->setOccupied(true); p2->setOccupied(true); circuitManager.addConnection(p1->getId(), p2->getId());
          { Pin* rp1 = resistor1.getRightPin(); Pin* rp2 = led1.getAnodePin(); Vector2f rs = rp1->getCenter();
          Vector2f re = rp2->getCenter(); WireConnection rc; Vector2f mid(re.x, rs.y);
          if (mid != rs) rc.segments.emplace_back(rs, mid); if (re != mid) rc.segments.emplace_back(mid, re);
          rc.pin1Id = rp1->getId(); rc.pin2Id = rp2->getId(); rc.color = Color::Green;
          for (auto& w : rc.segments) w.setColor(Color::Green); connections.push_back(rc); rp1->setOccupied(true); rp2->setOccupied(true); circuitManager.addConnection(rp1->getId(), rp2->getId()); }
          createRoutedWire(pGnd, led1.getCathodePin(), Color::Blue, 60.f);
        }
      }
      else {
        codeTextBox.setText(L"void setup() {\n  \n}\n\nvoid loop() {\n  \n}");
        if (pHigh && pLow && pGnd && p5V) {
          createRoutedWire(pLow, button1.getLeftPin(), Color::Yellow, 15.f); createRoutedWire(p5V, button1.getRightPin(), Color::Red, 0.f); createRoutedWire(pHigh, resistor1.getLeftPin(), Color::Magenta, 45.f);
          createAutoWire(resistor1.getRightPin(), led1.getAnodePin(), Color::Green); createRoutedWire(pGnd, led1.getCathodePin(), Color::Blue, 30.f);
        }
      }
      networkSimulator.resolveLogic(getAllPins(), circuitManager);
    }
    else if (activeLevel == 4) {
      string servoPinLabel = getPinLabel(genServoPin);
      wstring code = L"void setup() {\n  pinMode(" + wstring(servoPinLabel.begin(), servoPinLabel.end()) + L", OUTPUT);\n}\n\nvoid loop() {\n  servo.write(90);\n  delay(1200);\n}";
      codeTextBox.setText(code); codeTextBox.setEditable(false);
    }
    else { codeTextBox.setText(L"void setup() {\n}\n\nvoid loop() {\n  \n}"); codeTextBox.setEditable(true); }
}
```

Buttons.cpp:

```
#include "Buttons.hpp"

  Button::Button(Vector2f pos, Vector2f size, const Texture& tex, Color btnColor, Color hovColor)
    : sprite(tex), normalColor(btnColor), hoverColor(hovColor)
{
```

Buttons.hpp:

```
#pragma once

#include "Common.hpp"

class Button {
```

```
        sprite.setPosition(pos);
        Vector2u sizeT = tex.getSize();
        if (sizeT.x > 0 && sizeT.y > 0) {
            sprite.setScale({ size.x / sizeT.x, size.y / sizeT.y });
        }
        sprite.setColor(normalColor);
    }

void Button::setTexture(const Texture& tex, Vector2f size)
{
        sprite.setTexture(tex, true);
        Vector2u sizeT = tex.getSize();
        if (sizeT.x > 0 && sizeT.y > 0) {
            sprite.setScale({ size.x / sizeT.x, size.y / sizeT.y });
        }
    }

void Button::update(Vector2f mPos)
{
        if (sprite.getGlobalBounds().contains(mPos)) sprite.setColor(hoverColor);
        else sprite.setColor(normalColor);
    }

bool Button::checkClick(Vector2f m)
{
    return sprite.getGlobalBounds().contains(m);
}

void Button::draw(RenderWindow& w)
{ w.draw(sprite); }

FloatRect Button::getGlobalBounds() const
{ return sprite.getGlobalBounds(); }

    TransparentButton::TransparentButton(float x, float y, float w, float h)
{
        shape.setSize({ w, h });
        shape.setOrigin({ w / 2.f, h / 2.f });
        shape.setPosition({ x, y });
        shape.setFillColor(Color::Transparent);
    }

void TransparentButton::update(Vector2f mPos)
{
        if (shape.getGlobalBounds().contains(mPos)) {
            shape.setFillColor(Color(255, 255, 255, 40));
        }
        else {
            shape.setFillColor(Color::Transparent);
        }
    }

bool TransparentButton::checkClick(Vector2f mPos) const
{
    return shape.getGlobalBounds().contains(mPos);
}

void TransparentButton::draw(RenderWindow& w) const
{ w.draw(shape); }
```

CircuitManager.cpp:

```
#include "CircuitManager.hpp"

    const unordered_map<uint32_t, vector<uint32_t>>&
CircuitManager::getAdjList() const noexcept
{ return adjList; }

void CircuitManager::addConnection(uint32_t pin1Id, uint32_t pin2Id)
{
        auto& p1_conns = adjList[pin1Id];
        if (find(p1_conns.begin(), p1_conns.end(), pin2Id) == p1_conns.end()) {
            adjList[pin1Id].push_back(pin2Id);
            adjList[pin2Id].push_back(pin1Id);
        }
    }

void CircuitManager::removeConnection(uint32_t pin1Id, uint32_t pin2Id)
{
        auto& p1 = adjList[pin1Id];
        p1.erase(remove(p1.begin(), p1.end(), pin2Id), p1.end());
```

CircuitManager.hpp:

```
#pragma once

#include "Common.hpp"

class CircuitManager {

private:

    unordered_map<uint32_t, vector<uint32_t>> adjList;
public:
    const unordered_map<uint32_t, vector<uint32_t>>& getAdjList() const
noexcept;
    void addConnection(uint32_t pin1Id, uint32_t pin2Id);
    void removeConnection(uint32_t pin1Id, uint32_t pin2Id);
    void clearAll();
    vector<uint32_t> getConnectedNetwork(uint32_t startPinId);
};
```

```
    auto& p2 = adjList[pin2Id];
    p2.erase(remove(p2.begin(), p2.end(), pin1Id), p2.end());
  }

void CircuitManager::clearAll()
{ adjList.clear(); }

vector<uint32_t> CircuitManager::getConnectedNetwork(uint32_t startPinId)
{
    vector<uint32_t> network;
    unordered_set<uint32_t> visited;
    queue<uint32_t> q;
    q.push(startPinId); visited.insert(startPinId);
    while (!q.empty()) {
      uint32_t curr = q.front(); q.pop();
      network.push_back(curr);
      if (adjList.count(curr)) {
        for (uint32_t neighbor : adjList.at(curr)) {
          if (visited.find(neighbor) == visited.end()) {
            visited.insert(neighbor); q.push(neighbor);
          }
        }
      }
    }
    return network;
  }
```

ComponentNetwork.cpp:

```
#include "ComponentNetwork.hpp"

  void ComponentNetwork::resolveLogic(const vector<Pin*>& allPins,
CircuitManager& circuitManager)
{
    unordered_set<uint32_t> visited;
    unordered_map<uint32_t, Pin*> pinMap;
    shortCircuitDetected = false;
    for (Pin* p : allPins) {
      pinMap[p->getId()] = p;
      if (p->getRole() != PinRole::Source5V && p->getRole() !=
PinRole::Source3_3V && p->getRole() != PinRole::SourceGND) {
        p->setState(LogicLevel::Floating);
      }
    }
    for (Pin* pin : allPins) {
      if (visited.find(pin->getId()) != visited.end()) continue;
      vector<uint32_t> networkIds = circuitManager.getConnectedNetwork(pin-
>getId());
      bool hasHigh = false;
      bool hasGND = false;
      for (uint32_t id : networkIds) {
        Pin* p = pinMap[id];
        if (p->getRole() == PinRole::Source5V || p->getRole() ==
PinRole::Source3_3V) hasHigh = true;
        if (p->getRole() == PinRole::SourceGND) hasGND = true;
      }
      LogicLevel netLevel = LogicLevel::Floating;
      if (hasHigh) netLevel = LogicLevel::High;
      else if (hasGND) netLevel = LogicLevel::Low;
      for (uint32_t id : networkIds) {
        visited.insert(id);
        pinMap[id]->setState(netLevel);
      }
    }
    auto& adjList = circuitManager.getAdjList();
    for (Pin* startPin : allPins) {
      if (startPin->getRole() == PinRole::Source5V || startPin->getRole() ==
PinRole::Source3_3V) {
        queue<uint32_t> q;
        unordered_set<uint32_t> visitedNet;
        q.push(startPin->getId());
        visitedNet.insert(startPin->getId());
        bool gndReached = false;
        while (!q.empty()) {
          uint32_t curr = q.front(); q.pop();
          Pin* currPin = pinMap[curr];
          if (currPin && currPin->getRole() == PinRole::SourceGND) {
gndReached = true; break; }
          if (adjList.count(curr)) {
            for (uint32_t neighbor : adjList.at(curr)) {
```

ComponentNetwork.hpp:

```
#pragma once

#include "Common.hpp"
#include "Pin.hpp"
#include "CircuitManager.hpp"

class ComponentNetwork {

public:

  bool shortCircuitDetected = false;
  uint32_t resLeftId = 0;
  uint32_t resRightId = 0;    void resolveLogic(const vector<Pin*>& allPins,
CircuitManager& circuitManager);
};
```

```
                    if ((curr == resLeftId && neighbor == resRightId) || (curr ==
resRightId && neighbor == resLeftId)) continue;
                    if (visitedNet.find(neighbor) == visitedNet.end()) {
                        visitedNet.insert(neighbor);
                        q.push(neighbor);
                    }
                }
            }
        }
        if (gndReached) { shortCircuitDetected = true; break; }
    }
  }
}
```

Components.cpp:

```
#include "Components.hpp"

  void LED::applyScale()
{
    Vector2f size = sprite.getLocalBounds().size; if (size.x > 0.f && size.y > 0.f)
{
        sprite.setOrigin(size / 2.f);
        sprite.setScale({ 45.f / size.x, 75.f / size.y });
    }
  }

  LED::LED(Vector2f pos, const ResourceManager& rm) : offTex(rm.ledOffTex),
onTex(rm.ledOnTex), sprite(offTex), anodePin(pos + Vector2f(0.f, 50.f), rm),
cathodePin(pos + Vector2f(20.f, 40.f), rm), is_turned_on(false)
{
    sprite.setPosition(pos + Vector2f(20.f, 20.f)); applyScale();
anodePin.setRole(PinRole::Sink); cathodePin.setRole(PinRole::Sink);
  }

void LED::init()
{}

void LED::setTurnedOn(bool on)
{ is_turned_on = on; sprite.setTexture(on ? onTex : offTex, true); applyScale(); }

void LED::setBurnedOut(bool b)
{ is_burned_out = b; if (b) is_turned_on = false; }

bool LED::isBurnedOut() const noexcept
{ return is_burned_out; }

bool LED::isTurnedOn() const noexcept
{ return is_turned_on; }

void LED::draw(RenderWindow& w)
{
    if (is_burned_out) { sprite.setTexture(offTex, true); applyScale();
sprite.setColor(Color(70, 70, 70)); }
    else { sprite.setColor(Color::White); }
    w.draw(sprite); anodePin.draw(w); cathodePin.draw(w);
  }

Pin* LED::getAnodePin() noexcept
{ return &anodePin; }
Pin* LED::getCathodePin() noexcept
{ return &cathodePin; }

Resistor::Resistor(Vector2f pos, const ResourceManager& rm)
  : sprite(rm.resistorTex),
  leftPin(pos + Vector2f(15.f, -25.f), rm),
  rightPin(pos + Vector2f(15.f, 65.f), rm)
{
  Vector2f size = sprite.getLocalBounds().size;
  sprite.setOrigin(size / 2.f);
  sprite.setScale({ 30.f / size.x, 90.f / size.y });
  sprite.setPosition(pos + Vector2f(25.f, 30.f));
}

void Resistor::draw(RenderWindow& w)
{ w.draw(sprite); leftPin.draw(w); rightPin.draw(w); }

Pin* Resistor::getLeftPin() noexcept
{ return &leftPin; }
Pin* Resistor::getRightPin() noexcept
{ return &rightPin; }
```

Components.hpp:

```
#pragma once

#include "Common.hpp"
#include "Pin.hpp"
#include "ResourceManager.hpp"
#include "CircuitManager.hpp"
#include "ComponentNetwork.hpp"

class LED {

private:

  const Texture& offTex; const Texture& onTex; Sprite sprite;
  Pin anodePin; Pin cathodePin; bool is_turned_on;
  bool is_burned_out = false;    void applyScale();

public:
  LED(Vector2f pos, const ResourceManager& rm);
  void init();
  void setTurnedOn(bool on);
  void setBurnedOut(bool b);
  bool isBurnedOut() const noexcept;
  bool isTurnedOn() const noexcept;
  void draw(RenderWindow& w);
  Pin* getAnodePin() noexcept;
  Pin* getCathodePin() noexcept;
};

class Resistor {

private:

  Sprite sprite; Pin leftPin; Pin rightPin;
public:
  Resistor(Vector2f pos, const ResourceManager& rm);
  void draw(RenderWindow& w);
  Pin* getLeftPin() noexcept;
  Pin* getRightPin() noexcept;
};

class ServoComponent {

private:

  Sprite bodySprite;
  Sprite hornSprite;
  Pin hiddenGnd; Pin hiddenVcc; Pin hiddenSig;
  Pin gndPin; Pin vccPin; Pin signalPin;
  float currentAngle = 0.f;
  float targetAngle = 0.f;
  float rotationSpeed = 150.f;
  const Texture& bTex;
  const Texture& hTex;
  bool is_powered = false;
  Vector2f hornBasePos;

  void applyScale();
  void drawInternalWire(RenderWindow& w, Pin& p1, Pin& p2, Color c);

public:
  ServoComponent(Vector2f pos, const ResourceManager& rm);
  bool isConnectedCorrectly() const noexcept;
  void setPowered(bool p) noexcept;
  void setTargetAngle(float angle) noexcept;
  float getCurrentAngle() const noexcept;
```

```
   void ServoComponent::applyScale()
{
      Vector2f bSize = bodySprite.getLocalBounds().size;
      if (bSize.x > 0.f && bSize.y > 0.f) {
         bodySprite.setOrigin(bSize / 2.f); bodySprite.setScale({ 80.f / bSize.x, 120.f
/ bSize.y });
      }
      Vector2f hSize = hornSprite.getLocalBounds().size;
      if (hSize.x > 0.f && hSize.y > 0.f) {
         hornSprite.setOrigin({ 21.5f, hSize.y / 2.f });hornSprite.setScale({ 100.f /
hSize.x, 24.f / hSize.y });
      }
   }

void ServoComponent::drawInternalWire(RenderWindow& w, Pin& p1, Pin& p2,
Color c)
{
      Vector2f s = p1.getCenter(); Vector2f e = p2.getCenter(); Vector2f d = e - s;
float len = hypot(d.x, d.y);
      RectangleShape line({ len, 4.f }); line.setOrigin({ 0.f, 2.f });
line.setPosition(s);
      line.setRotation(degrees(atan2(d.y, d.x) * 180.f / 3.14159f));
line.setFillColor(c); w.draw(line);
   }

ServoComponent::ServoComponent(Vector2f pos, const ResourceManager& rm)
   : bodySprite(rm.servoBodyTex), hornSprite(rm.servoHornTex),
   hiddenGnd(pos + Vector2f(-25.f, 40.f), rm), hiddenVcc(pos + Vector2f(-10.f,
40.f), rm), hiddenSig(pos + Vector2f(5.f, 40.f), rm),
   gndPin(pos + Vector2f(-25.f, 85.f), rm), vccPin(pos + Vector2f(-10.f, 85.f), rm),
signalPin(pos + Vector2f(5.f, 85.f), rm),
   bTex(rm.servoBodyTex), hTex(rm.servoHornTex), hornBasePos(pos +
Vector2f(0.f, -10.f))
{
   applyScale();
   bodySprite.setPosition(pos);
   hornSprite.setPosition(hornBasePos);
   hiddenGnd.setVisible(false); hiddenGnd.setOccupied(true);
   hiddenVcc.setVisible(false); hiddenVcc.setOccupied(true);
   hiddenSig.setVisible(false); hiddenSig.setOccupied(true);
   gndPin.setRole(PinRole::Sink);                vccPin.setRole(PinRole::Sink);
signalPin.setRole(PinRole::Sink);
}

bool ServoComponent::isConnectedCorrectly() const noexcept
{
      return is_powered;
   }

void ServoComponent::setPowered(bool p) noexcept
{ is_powered = p; }

void ServoComponent::setTargetAngle(float angle) noexcept
{ targetAngle = clamp(angle, 0.f, 180.f); }

float ServoComponent::getCurrentAngle() const noexcept
{ return currentAngle; }

float ServoComponent::getTargetAngle() const noexcept
{ return targetAngle; }

void ServoComponent::reset() noexcept
{
   currentAngle = 0.f; targetAngle = 0.f; hornSprite.setRotation(degrees(0.f));
hornSprite.setPosition(hornBasePos);
}

void ServoComponent::update(float dt)
{
      if (is_powered) {
         if (currentAngle < targetAngle) { currentAngle += rotationSpeed * dt; if
(currentAngle > targetAngle) currentAngle = targetAngle; }
         else if (currentAngle > targetAngle) { currentAngle -= rotationSpeed * dt;
if (currentAngle < targetAngle) currentAngle = targetAngle; }
      }
      hornSprite.setRotation(degrees(-currentAngle));
   }
   float getTargetAngle() const noexcept;
   void reset() noexcept;
   void update(float dt);
   void draw(RenderWindow& w);
   Pin* getGndPin() noexcept;
   Pin* getVccPin() noexcept;
   Pin* getSignalPin() noexcept;
};

class ArduinoButton {

private:

   const Texture& offTex;
   const Texture& onTex;
   Sprite sprite;
   Pin leftPin;
   Pin rightPin;
   bool is_pressed = false;    void applyScale();

public:
   ArduinoButton(Vector2f pos, const ResourceManager& rm);
   bool checkClick(Vector2f m);
   void updateSpriteState();
   void setPressed(bool pressed, CircuitManager& cm, const vector<Pin*>&
allPins, ComponentNetwork& net);
   void reset();
   bool isPressed() const noexcept;
   void draw(RenderWindow& w);
   Pin* getLeftPin() noexcept;
   Pin* getRightPin() noexcept;
};
```

```
void ServoComponent::draw(RenderWindow& w)
{
    drawInternalWire(w, hiddenGnd, gndPin, Color(139, 69, 19)); drawInternalWire(w, hiddenVcc, vccPin, Color::Red); drawInternalWire(w, hiddenSig, signalPin, Color(255, 140, 0));
    if (bTex.getSize().x == 0) {
        RectangleShape rect({ 120.f, 80.f }); rect.setFillColor(Color(0, 80, 180)); rect.setOrigin({ 60.f, 40.f }); rect.setPosition(bodySprite.getPosition()); w.draw(rect);
    }
    else { w.draw(bodySprite); }
    if (hTex.getSize().x == 0) {
        RectangleShape rect({ 100.f, 24.f }); rect.setFillColor(Color(220, 220, 220)); rect.setOrigin({ 50.f, 12.f }); rect.setPosition(hornSprite.getPosition()); rect.setRotation(degrees(currentAngle)); w.draw(rect);
        CircleShape pointer(5.f); pointer.setFillColor(Color::Red); pointer.setOrigin({ 5.f, 5.f });
        float rad = currentAngle * 3.14159f / 180.f; pointer.setPosition(hornSprite.getPosition() + Vector2f(cos(rad) * 40.f, sin(rad) * 40.f)); w.draw(pointer);
    }
    else { w.draw(hornSprite); }
    hiddenGnd.draw(w); hiddenVcc.draw(w); hiddenSig.draw(w); gndPin.draw(w); vccPin.draw(w); signalPin.draw(w);
}

Pin* ServoComponent::getGndPin() noexcept
{ return &gndPin; }
Pin* ServoComponent::getVccPin() noexcept
{ return &vccPin; }
Pin* ServoComponent::getSignalPin() noexcept
{ return &signalPin; }

void ArduinoButton::applyScale()
{
    Vector2f size = sprite.getLocalBounds().size;
    if (size.x > 0.f && size.y > 0.f) {
        sprite.setOrigin(size / 2.f);
        sprite.setScale({ 50.f / size.x, 50.f / size.y });
    }
}

ArduinoButton::ArduinoButton(Vector2f pos, const ResourceManager& rm)
    : offTex(rm.buttonOffTex), onTex(rm.buttonOnTex), sprite(offTex),
    leftPin(pos + Vector2f(-10.5f, 35.f), rm), rightPin(pos + Vector2f(20.f, 35.f), rm)
{
    sprite.setPosition(pos + Vector2f(15.f, 15.f));
    applyScale();
    leftPin.setRole(PinRole::Normal);
    rightPin.setRole(PinRole::Normal);
}

bool ArduinoButton::checkClick(Vector2f m)
{ return sprite.getGlobalBounds().contains(m); }

void ArduinoButton::updateSpriteState()
{
    bool shouldGlow = is_pressed && (leftPin.getState() == LogicLevel::High || rightPin.getState() == LogicLevel::High);
    sprite.setTexture(shouldGlow ? onTex : offTex, true);
    applyScale();
}

void ArduinoButton::setPressed(bool pressed, CircuitManager& cm, const vector<Pin*>& allPins, ComponentNetwork& net)
{
    if (is_pressed != pressed) {
        is_pressed = pressed;
        if (is_pressed) { cm.addConnection(leftPin.getId(), rightPin.getId()); }
        else { cm.removeConnection(leftPin.getId(), rightPin.getId()); }
        net.resolveLogic(allPins, cm);
        updateSpriteState();
    }
}

void ArduinoButton::reset()
{ is_pressed = false; sprite.setTexture(offTex, true); applyScale(); }
```

```
bool ArduinoButton::isPressed() const noexcept
{ return is_pressed; }

void ArduinoButton::draw(RenderWindow& w)
{ w.draw(sprite); leftPin.draw(w); rightPin.draw(w); }

Pin* ArduinoButton::getLeftPin() noexcept
{ return &leftPin; }

Pin* ArduinoButton::getRightPin() noexcept
{ return &rightPin; }
```

ContextMenu.cpp:

```
#include "ContextMenu.hpp"

   ContextMenu::ContextMenu(const Font& font)
      : txtColor(font, L"Изменить цвет", 14), txtDelete(font, L"Удалить провод",
14)
{
      bgRect.setFillColor(Color(50, 50, 50, 240));
      bgRect.setOutlineThickness(1.f); bgRect.setOutlineColor(Color::White);
      btnColorRect.setFillColor(Color(70, 70, 70));
      btnDeleteRect.setFillColor(Color(200, 50, 50));
      txtColor.setFillColor(Color::White);
      txtDelete.setFillColor(Color::White);
      for (int i = 0; i < 5; ++i) {
         RectangleShape sq({ 30.f, 30.f });
         sq.setFillColor(palette[i]);
         sq.setOutlineThickness(1.f); sq.setOutlineColor(Color::White);
         colorSquares.push_back(sq);
      }
   }

void ContextMenu::open(Vector2f pos, int wireIdx)
{
      currentState = Main; activeWireIndex = wireIdx;
      bgRect.setSize({ 150.f, 60.f }); bgRect.setPosition(pos);
      btnColorRect.setSize({ 140.f, 25.f }); btnColorRect.setPosition({ pos.x + 5.f,
pos.y + 5.f }); txtColor.setPosition({ pos.x + 10.f, pos.y + 8.f });
      btnDeleteRect.setSize({ 140.f, 25.f }); btnDeleteRect.setPosition({ pos.x + 5.f,
pos.y + 30.f }); txtDelete.setPosition({ pos.x + 10.f, pos.y + 33.f });
   }

void ContextMenu::openColors()
{
      currentState = Colors; bgRect.setSize({ 180.f, 40.f });
      for (int i = 0; i < 5; ++i) colorSquares[i].setPosition({ bgRect.getPosition().x +
5.f + i * 35.f, bgRect.getPosition().y + 5.f });
   }

void ContextMenu::close()
{ currentState = Closed; activeWireIndex = -1; }

bool ContextMenu::handleLeftClick(Vector2f mPos, vector<WireConnection>&
connections, CircuitManager& cm, const vector<Pin*>& allPins,
ComponentNetwork& net, function<void()> onWireDeleted)
{
      if (currentState == Closed) return false;
      if (currentState == Main) {
         if (btnColorRect.getGlobalBounds().contains(mPos)) { openColors();
return true; }
         if (btnDeleteRect.getGlobalBounds().contains(mPos)) {
            if (activeWireIndex >= 0 && activeWireIndex < connections.size()) {
               auto& conn = connections[activeWireIndex];
               cm.removeConnection(conn.pin1Id, conn.pin2Id);
               for (Pin* p : allPins) { if (p->getId() == conn.pin1Id || p->getId() ==
conn.pin2Id) p->setOccupied(false); }
               connections.erase(connections.begin() + activeWireIndex);
               net.resolveLogic(allPins, cm);
               if (onWireDeleted) onWireDeleted();
            }
            close(); return true;
         }
      }
      else if (currentState == Colors) {
         for (int i = 0; i < 5; ++i) {
            if (colorSquares[i].getGlobalBounds().contains(mPos)) {
               if (activeWireIndex >= 0 && activeWireIndex < connections.size())
connections[activeWireIndex].setColor(palette[i]);
               close(); return true;
```

ContextMenu.hpp:

```
# pragma once

#include "Common.hpp"
#include "Pin.hpp"
#include "CircuitManager.hpp"
#include "ComponentNetwork.hpp"
#include "Wire.hpp"

class ContextMenu {

public:

   enum State { Closed, Main, Colors };
   State currentState = Closed;
   int activeWireIndex = -1;
private:

   RectangleShape bgRect, btnColorRect, btnDeleteRect;
   Text txtColor, txtDelete;
   vector<RectangleShape> colorSquares;
   vector<Color> palette = { Color::Red, Color::Green, Color::Blue,
Color::Yellow, Color::Magenta };
public:
   ContextMenu(const Font& font);
   void open(Vector2f pos, int wireIdx);
   void openColors();
   void close();
   bool handleLeftClick(Vector2f mPos, vector<WireConnection>&
connections, CircuitManager& cm, const vector<Pin*>& allPins,
ComponentNetwork& net, function<void()> onWireDeleted = nullptr);
   void draw(RenderWindow& w);
};
```

```
            }
        }
    }
    close(); return false;
}

void ContextMenu::draw(RenderWindow& w)
{
    if (currentState == Closed) return;
    w.draw(bgRect);
    if (currentState == Main) { w.draw(btnColorRect); w.draw(txtColor);
w.draw(btnDeleteRect); w.draw(txtDelete); }
    else if (currentState == Colors) { for (auto& sq : colorSquares) w.draw(sq); }
}
```

Globals.cpp:

```
#include "Globals.hpp"

mutex boardMutex;
atomic<bool> isScriptRunning{ false };
atomic<bool> stopRequested{ false };

int activeLevel = 0;
int ledOnCount = 0;
bool ledWasOn = false;
float victoryTimer = 0.f;
bool victoryTriggered = false;
float victoryDelayTimer = 0.f;
bool levelCleared = false;
int genPinHigh = 13;
int genPinLow = 18;
int genServoPin = 9;
bool btnAsSwitchLevel3 = false;
bool btnAsSwitchLevel4 = false;
```

Globals.hpp:

```
#pragma once

#include "Common.hpp"

extern mutex boardMutex;
extern atomic<bool> isScriptRunning;
extern atomic<bool> stopRequested;

extern int activeLevel;
extern int ledOnCount;
extern bool ledWasOn;
extern float victoryTimer;
extern bool victoryTriggered;
extern float victoryDelayTimer;
extern bool levelCleared;
extern int genPinHigh;
extern int genPinLow;
extern int genServoPin;
extern bool btnAsSwitchLevel3;
extern bool btnAsSwitchLevel4;
```

Helpers.cpp:

```
#include "Helpers.hpp"

float getDistanceToSegment(Vector2f p, Vector2f a, Vector2f b) {
    float dx = a.x - b.x;
    float dy = a.y - b.y;
    float l2 = dx * dx + dy * dy;
    if (l2 == 0.0f) return hypot(p.x - a.x, p.y - a.y);
    float t = max(0.0f, min(1.0f, ((p.x - a.x) * (b.x - a.x) + (p.y - a.y) * (b.y - a.y)) /
l2));
    Vector2f projection = a + t * (b - a);
    return hypot(p.x - projection.x, p.y - projection.y);
}
```

Helpers.hpp:

```
#pragma once

#include "Common.hpp"

inline void replaceAll(string& str, const string& from, const string& to) {
    if (from.empty()) return;
    size_t start_pos = 0;
    while ((start_pos = str.find(from, start_pos)) != string::npos) {
        str.replace(start_pos, from.length(), to);
        start_pos += to.length();
    }
}

float getDistanceToSegment(Vector2f p, Vector2f a, Vector2f b);
```

LevelCarousel.cpp:

```
#include "LevelCarousel.hpp"

    LevelCarousel::LevelCarousel(const Font& font, const ResourceManager& rm)
{
    for (int i = 0; i < totalLevels; ++i) {
        RectangleShape box({ 400.f, 600.f });
        box.setFillColor(Color::White);
        box.setTexture(&rm.levelTextures[i]);
        box.setOutlineThickness(4.f);
        box.setOutlineColor(Color::White);
        levelBoxes.push_back(box);
    }
}

void LevelCarousel::reset()
{ currentIdx = 0; currentOffset = 0.f; targetOffset = 0.f; }

void LevelCarousel::moveLeft()
{ if (currentIdx > 0) { currentIdx--; targetOffset += 500.f; } }

void LevelCarousel::moveRight()
{ if (currentIdx < totalLevels - 1) { currentIdx++; targetOffset -= 500.f; } }

int LevelCarousel::getCurrentIdx() const noexcept
{ return currentIdx; }
```

LevelCarousel.hpp:

```
#pragma once

#include "Common.hpp"
#include "ResourceManager.hpp"

class LevelCarousel {

private:

    vector<RectangleShape> levelBoxes;
    int currentIdx = 0;
    int totalLevels = 6;
    float currentOffset = 0.f;
    float targetOffset = 0.f;
    Clock clickClock;
    int lastClickedIdx = -1;
public:
    LevelCarousel(const Font& font, const ResourceManager& rm);
    void reset();
    void moveLeft();
    void moveRight();
    int getCurrentIdx() const noexcept;
    void update(float dt);
    int checkClick(Vector2f mPos);
    void draw(RenderWindow& w);
};
```

```
void LevelCarousel::update(float dt)
{
    currentOffset += (targetOffset - currentOffset) * 10.f * dt;
    float startX = 960.f - 200.f;
    float startY = 240.f;
    for (int i = 0; i < totalLevels; ++i) {
        float xPos = startX + (i * 500.f) + currentOffset;
        levelBoxes[i].setPosition({ xPos, startY });
        if (i == currentIdx) levelBoxes[i].setOutlineColor(Color::Cyan);
        else levelBoxes[i].setOutlineColor(Color(100, 100, 100));
    }
}

int LevelCarousel::checkClick(Vector2f mPos)
{
    for (int i = 0; i < totalLevels; ++i) {
        if (levelBoxes[i].getGlobalBounds().contains(mPos)) {
            if (i == currentIdx) {
                if (lastClickedIdx == i && clickClock.getElapsedTime().asMilliseconds() < 400) return i;
                lastClickedIdx = i; clickClock.restart();
            }
            else {
                if (i > currentIdx) moveRight();
                else moveLeft();
            }
            return -1;
        }
    }
    return -1;
}

void LevelCarousel::draw(RenderWindow& w)
{ for (int i = 0; i < totalLevels; ++i) w.draw(levelBoxes[i]); }
```

Pin.cpp:

```
#include "Pin.hpp"

uint32_t Pin::global_id_counter = 1;

Pin::Pin(Vector2f pos, const ResourceManager& rm)
    : id(global_id_counter++), nTex(rm.pinNormalTex), sTex(rm.pinSelectedTex), sprite(nTex), is_selected(false)
{
    if (nTex.getSize().x == 0) return;
    Vector2f size = sprite.getLocalBounds().size;
    sprite.setOrigin(size / 2.f);
    sprite.setScale({ 15.f / size.x, 15.f / size.y });
    sprite.setPosition(pos + Vector2f(10.f, 10.f));
}

uint32_t Pin::getId() const noexcept
{ return id; }

bool Pin::checkClick(Vector2f m)
{ return is_visible && sprite.getGlobalBounds().contains(m); }

void Pin::draw(RenderWindow& w)
{ if (is_visible) w.draw(sprite); }

void Pin::setSelected(bool s)
{ is_selected = s; sprite.setTexture(s ? sTex : nTex); }

Vector2f Pin::getCenter()
{ return sprite.getPosition(); }

bool Pin::IsSelected() const noexcept
{ return is_selected; }

bool Pin::isOccupied() const noexcept
{ return is_occupied; }

void Pin::setOccupied(bool status)
{ is_occupied = status; }

bool Pin::getVisible() const noexcept
{ return is_visible; }

void Pin::setVisible(bool v)
{ is_visible = v; }
```

Pin.hpp:

```
#pragma once

#include "Common.hpp"
#include "ResourceManager.hpp"

class Pin {

private:
    static uint32_t global_id_counter;

    uint32_t id;
    const Texture& nTex;
    const Texture& sTex;
    Sprite sprite;
    bool is_selected;
    bool is_occupied = false;
    bool is_visible = true;
    int arduinoPinNum = -1;
    PinRole role = PinRole::Normal;
    LogicLevel state = LogicLevel::Floating;
    function<void(LogicLevel)> onStateChange = nullptr;
public:
    Pin(Vector2f pos, const ResourceManager& rm);
    uint32_t getId() const noexcept;
    bool checkClick(Vector2f m);
    void draw(RenderWindow& w);
    void setSelected(bool s);
    Vector2f getCenter();
    bool IsSelected() const noexcept;
    bool isOccupied() const noexcept;
    void setOccupied(bool status);
    bool getVisible() const noexcept;
    void setVisible(bool v);
    void setArduinoPin(int num);
    int getArduinoPin() const noexcept;
    void setRole(PinRole r);
    PinRole getRole() const noexcept;
    void setState(LogicLevel s);
    LogicLevel getState() const noexcept;
    void setCallback(function<void(LogicLevel)> cb);
};
```

```
void Pin::setArduinoPin(int num)
{ arduinoPinNum = num; }

int Pin::getArduinoPin() const noexcept
{ return arduinoPinNum; }

void Pin::setRole(PinRole r)
{ role = r; }

PinRole Pin::getRole() const noexcept
{ return role; }

void Pin::setState(LogicLevel s)
{
      if (state != s) {
         state = s;
         if (onStateChange) onStateChange(s);
      }
   }

LogicLevel Pin::getState() const noexcept
{ return state; }

void Pin::setCallback(function<void(LogicLevel)> cb)
{ onStateChange = cb; }
```

ResourceManager.cpp:

```
#include "ResourceManager.hpp"

ResourceManager* g_resourceManager = nullptr;
bool g_soundEnabled = true;

   ResourceManager::ResourceManager()
      : clickSnd(clickBuf), errorSnd(errorBuf), victorySnd(victoryBuf), stopSnd(stopBuf)
{
      g_resourceManager = this;

      if (!pinNormalTex.loadFromFile("images/components/pin_1.1.png")) cerr << "Error: pin\n";
      if (!pinSelectedTex.loadFromFile("images/components/selected_pin.png")) cerr << "Error: selected_pin\n";
      if (!boardTex.loadFromFile("images/components/arduino_board_1.1.png")) { cerr << "Error: board\n"; hasBoardTexture = false; }
      if (!resistorTex.loadFromFile("images/components/resistor.png")) cerr << "Error: resistor\n";
      if (!ledOffTex.loadFromFile("images/components/led_off_1.png")) cerr << "Error: led_off\n";
      if (!ledOnTex.loadFromFile("images/components/led_on_1.png")) cerr << "Error: led_on\n";
      if (!buttonOffTex.loadFromFile("images/components/button_off.png")) cerr << "Error: button_off\n";
      if (!buttonOnTex.loadFromFile("images/components/button_on.png")) cerr << "Error: button_on\n";
      if (!servoBodyTex.loadFromFile("images/components/servo_body.png")) cerr << "Error: servo_body\n";
      if (!servoHornTex.loadFromFile("images/components/servo_horn.png")) cerr << "Error: servo_horn\n";
      if (!levelBgTex.loadFromFile("images/background/level_bg1.png")) cerr << "Error: level_bg\n";
      if (!font.openFromFile("undertale_battle_font.ttf")) cerr << "Error: font\n";
      levelTextures.resize(6);
      for (int i = 0; i < 6; ++i) {
         string path = "images/levels/level" + to_string(i) + ".png";
         if (!levelTextures[i].loadFromFile(path)) cerr << "Error: " << path << "\n";
      }
      theoryTextures.resize(11);
      for (int i = 0; i < 11; ++i) {
         string path = "images/theory/S" + to_string(i+1) + ".JPG";
         if (!theoryTextures[i].loadFromFile(path));
      }
      if (!btnPracticeTex.loadFromFile("images/buttons/practice.png")) cerr << "Error: practice.png\n";
      if (!btnSettingsTex.loadFromFile("images/buttons/settings.png")) cerr << "Error: settings.png\n";
      if (!btnExitTex.loadFromFile("images/buttons/exit.png")) cerr << "Error: exit.png\n";
      if (!btnTheoryTex.loadFromFile("images/buttons/theory.png")) btnTheoryTex = btnPracticeTex;
```

ResourceManager.hpp:

```
#pragma once

#include "Common.hpp"

class ResourceManager {

public:


   Texture pinNormalTex, pinSelectedTex, boardTex;
   Texture resistorTex, ledOffTex, ledOnTex;
   Texture buttonOffTex, buttonOnTex;
   Texture servoBodyTex, servoHornTex;
   Texture levelBgTex;
   vector<Texture> levelTextures;
   vector<Texture> theoryTextures;
   Font font;
   bool hasBoardTexture = true;
   Texture btnTheoryTex, btnPracticeTex, btnSettingsTex, btnExitTex, btnHelpTex;
   Texture btnFsOnTex, btnFsOffTex;
   Texture btnSoundOnTex, btnSoundOffTex;
   Texture btnBackTex, btnStartTex, btnCloseTex;
   Texture btnRunTex, btnStopTex, btnResetTex, btnSimSettingsTex, btnSimResetTaskTex;
   Texture btnRestartTex, btnMenuGoTex, btnReturnTex, btnToLevelsTex, btnNextLevelTex;
   Texture btnConditionUpTex, btnConditionDownTex;
   Texture arrowLeftTex, arrowRightTex;
   Texture errorCodTex, errorDiodTex, errorShortTex, victoryBgTex;
   Texture menuBgTex;


   SoundBuffer clickBuf, errorBuf, victoryBuf, stopBuf;
   Sound clickSnd, errorSnd, victorySnd, stopSnd;

   ResourceManager();

   void playClick();
   void playError();
   void playVictory();
   void playStop();
};

extern ResourceManager* g_resourceManager;
extern bool g_soundEnabled;
```

```
    if (!btnHelpTex.loadFromFile("images/buttons/help.png")) btnHelpTex =
btnSettingsTex;
    if (!btnFsOnTex.loadFromFile("images/buttons/fs_off.png")) cerr << "Error:
fs_on.png\n";
    if (!btnFsOffTex.loadFromFile("images/buttons/fs_on.png")) cerr << "Error:
fs_off.png\n";
    if (!btnSoundOnTex.loadFromFile("images/buttons/sound_off.png")) cerr <<
"Error: sound_on.png\n";
    if (!btnSoundOffTex.loadFromFile("images/buttons/sound_on.png")) cerr <<
"Error: sound_off.png\n";
    if (!btnBackTex.loadFromFile("images/buttons/back.png")) cerr << "Error:
back.png\n";
    if (!btnStartTex.loadFromFile("images/buttons/start.png")) cerr << "Error:
start.png\n";
    if (!btnRunTex.loadFromFile("images/buttons/run.png")) cerr << "Error:
run.png\n";
    if (!btnStopTex.loadFromFile("images/buttons/stop.png")) cerr << "Error:
stop.png\n";
    if (!btnResetTex.loadFromFile("images/buttons/reset.png")) cerr << "Error:
reset.png\n";
    if (!btnSimSettingsTex.loadFromFile("images/buttons/settings.png")) cerr <<
"Error: sim_settings.png\n";
    if
(!btnSimResetTaskTex.loadFromFile("images/buttons/sim_reset_task.png")) cerr
<< "Error: sim_reset_task.png\n";
    if (!btnRestartTex.loadFromFile("images/buttons/restart.png")) cerr <<
"Error: restart.png\n";
    if (!btnMenuGoTex.loadFromFile("images/buttons/menu_go.png")) cerr <<
"Error: menu_go.png\n";
    if (!btnToLevelsTex.loadFromFile("images/buttons/to_levels.png")) cerr <<
"Error: to_levels.png\n";
    if (!btnNextLevelTex.loadFromFile("images/buttons/next_level.png")) cerr
<< "Error: next_level.png\n";
    if (!btnConditionUpTex.loadFromFile("images/buttons/condition_up.png"))
btnConditionUpTex = btnSettingsTex;
    if
(!btnConditionDownTex.loadFromFile("images/buttons/condition_down.png"))
btnConditionDownTex = btnSettingsTex;
    if (!arrowLeftTex.loadFromFile("images/buttons/arrow_left.png"))
arrowLeftTex = btnBackTex;
    if (!arrowRightTex.loadFromFile("images/buttons/arrow_right.png"))
arrowRightTex = btnBackTex;
    if (!errorCodTex.loadFromFile("images/buttons/error_cod.png")) cerr <<
"Error: error_cod.png не найден\n";
    if (!errorDiodTex.loadFromFile("images/buttons/error_diod.png")) cerr <<
"Error: error_diod.png не найден\n";
    if (!errorShortTex.loadFromFile("images/buttons/UNO_error.png")) cerr <<
"Error: UNO_error.png не найден\n";
    if (!victoryBgTex.loadFromFile("images/buttons/level_completed_1.png"))
cerr << "Error: Окно победы не найдено\n";
    if (!menuBgTex.loadFromFile("images/background/main_bg.png")) {
menuBgTex = levelBgTex;}

    if (!errorBuf.loadFromFile("sounds/error.mp3")) cerr << "Error:
sounds/error.wav не найден\n";
    if (!victoryBuf.loadFromFile("sounds/victory.mp3")) cerr << "Error:
sounds/victory.wav не найден\n";
    if (!stopBuf.loadFromFile("sounds/stop.mp3")) cerr << "Error:
sounds/stop.wav не найден\n";
  }

void ResourceManager::playClick()
{
  if (!g_soundEnabled) return;
  clickSnd.stop(); clickSnd.play();
}

void ResourceManager::playError()
{
  if (!g_soundEnabled) return;
  errorSnd.stop(); errorSnd.play();
}

void ResourceManager::playVictory()
{
  if (!g_soundEnabled) return;
  victorySnd.stop(); victorySnd.play();
}
```

```
void ResourceManager::playStop()
{
   if (!g_soundEnabled) return;
   stopSnd.stop(); stopSnd.play();
}
```

TextBox.cpp:

```
#include "TextBox.hpp"

   void TextBox::updateText()
{
      sfText.setString(inputString);
      cursorBlinkClock.restart();
      cursorVisible = true;
      scrollToCursor();
   }

void TextBox::scrollToCursor()
{
      if (inputString.isEmpty()) { scrollX = 0.f; scrollY = 0.f; return; }
      sfText.setPosition({ 10.f, 10.f });
      Vector2f cPos = sfText.findCharacterPos(cursorPos);
      float charHeight = sfText.getCharacterSize() + 4.f;
      if (cPos.x < scrollX + 10.f) scrollX = max(0.f, cPos.x - 20.f);
      else if (cPos.x > scrollX + canvas.getSize().x - 20.f) scrollX = cPos.x -
canvas.getSize().x + 20.f;
      if (cPos.y < scrollY + 10.f) scrollY = max(0.f, cPos.y - charHeight);
      else if (cPos.y + charHeight > scrollY + canvas.getSize().y - 10.f) scrollY =
cPos.y + charHeight - canvas.getSize().y + 10.f;
   }

void TextBox::clampScroll()
{
      float contentHeight = sfText.getLocalBounds().size.y + 50.f;
      float maxScrollY = contentHeight > canvas.getSize().y ? contentHeight -
canvas.getSize().y : 0.f;
      if (scrollY < 0.f) scrollY = 0.f;
      if (scrollY > maxScrollY) scrollY = maxScrollY;
      float contentWidth = sfText.getLocalBounds().size.x + 20.f;
      float maxScrollX = contentWidth > canvas.getSize().x ? contentWidth -
canvas.getSize().x : 0.f;
      if (scrollX < 0.f) scrollX = 0.f;
      if (scrollX > maxScrollX) scrollX = maxScrollX;
   }

FloatRect TextBox::getVScrollRect() const
{
      float contentHeight = sfText.getLocalBounds().size.y + 50.f;
      if (contentHeight <= canvas.getSize().y) return FloatRect({ 0.f, 0.f }, { 0.f, 0.f
});
      float ratio = canvas.getSize().y / contentHeight;
      float barHeight = max(20.f, canvas.getSize().y * ratio);
      float scrollRatio = scrollY / (contentHeight - canvas.getSize().y);
      float barY = rect.getPosition().y + scrollRatio * (canvas.getSize().y -
barHeight);
      return FloatRect({ rect.getPosition().x + canvas.getSize().x - 12.f, barY }, {
12.f, barHeight });
   }

FloatRect TextBox::getHScrollRect() const
{
      float contentWidth = sfText.getLocalBounds().size.x + 20.f;
      if (contentWidth <= canvas.getSize().x) return FloatRect({ 0.f, 0.f }, { 0.f, 0.f
});
      float ratio = canvas.getSize().x / contentWidth;
      float barWidth = max(20.f, canvas.getSize().x * ratio);
      float scrollRatio = scrollX / (contentWidth - canvas.getSize().x);
      float barX = rect.getPosition().x + scrollRatio * (canvas.getSize().x -
barWidth);
      return FloatRect({ barX, rect.getPosition().y + canvas.getSize().y - 12.f }, {
barWidth, 12.f });
   }

bool TextBox::deleteSelection()
{
      if (!is_editable) return false;
      if (cursorPos == selectAnchor) return false;
      size_t start = min(cursorPos, selectAnchor);
      size_t end = max(cursorPos, selectAnchor);
      inputString.erase(start, end - start);
```

TextBox.hpp:

```
#pragma once

#include "Common.hpp"

class TextBox {

private:

   RectangleShape rect;
   Text sfText;
   String inputString;
   bool is_focused = false;
   size_t cursorPos = 0;
   size_t selectAnchor = 0;
   bool isDragging = false;
   bool isDraggingVScroll = false;
   bool isDraggingHScroll = false;
   Clock cursorBlinkClock;
   bool cursorVisible = true;
   bool is_editable = true;
   function<void()> onTextChange = nullptr;
   size_t preferredColumn = 0;
   bool updatePreferredColumn = true;
   RenderTexture canvas;
   Sprite canvasSprite;
   float scrollX = 0.f;
   float scrollY = 0.f;    void updateText();
   void scrollToCursor();
   void clampScroll();
   FloatRect getVScrollRect() const;
   FloatRect getHScrollRect() const;
   bool deleteSelection();
   void insertText(const String& str);
   size_t getIndexAtPos(Vector2f pos);

public:
   TextBox(Vector2f pos, Vector2f size, const Font& font);
   FloatRect getGlobalBounds() const;
   void setFocused(bool f);
   bool isFocused() const noexcept;
   bool checkClick(Vector2f m);
   void setOnChangeCallback(function<void()> cb);
   void setEditable(bool editable);
   void handleMouseWheel(float delta, bool shiftPressed);
   void setText(const wstring& str);
   string getText() const;
   void handleTextEntered(char32_t unicode);
   void handleKeyPressed(Keyboard::Key key, bool ctrl, bool shift);
   void handleMousePress(Vector2f pos);
   void handleMouseMove(Vector2f pos);
   void handleMouseRelease();
   void draw(RenderWindow& w);
};
```

```
        cursorPos = selectAnchor = start; updateText(); return true;
    }

void TextBox::insertText(const String& str)
{
        if (!is_editable) return;
        deleteSelection();
        inputString.insert(cursorPos, str);
        cursorPos += str.getSize();
        selectAnchor = cursorPos;
        updateText();
    }

size_t TextBox::getIndexAtPos(Vector2f pos)
{
        Vector2f localPos = pos - rect.getPosition() + Vector2f(scrollX, scrollY);
        float minDist = 999999.f; size_t bestIdx = inputString.getSize();
        float charHeight = sfText.getCharacterSize() + 4.f;
        sfText.setPosition({ 10.f, 10.f });
        for (size_t i = 0; i <= inputString.getSize(); ++i) {
            Vector2f p = sfText.findCharacterPos(i);
            float d = abs((p.y + charHeight / 2.f) - localPos.y) * 1000.f + abs(p.x -
localPos.x);
            if (d < minDist) { minDist = d; bestIdx = i; }
        }
        return bestIdx;
    }

    TextBox::TextBox(Vector2f pos, Vector2f size, const Font& font)
        : sfText(font, ""),
        canvas(
{ static_cast<unsigned int>(size.x), static_cast<unsigned int>(size.y) }),
        canvasSprite(canvas.getTexture())
    {
        rect.setPosition(pos); rect.setSize(size); rect.setFillColor(Color(30, 30, 30));
        rect.setOutlineThickness(2.f); rect.setOutlineColor(Color(100, 100, 100));
        sfText.setCharacterSize(24); sfText.setFillColor(Color::Green);
        sfText.setPosition({ 10.f, 10.f });
        canvasSprite.setPosition(pos);
        cursorPos = selectAnchor = inputString.getSize(); updateText();
    }

FloatRect TextBox::getGlobalBounds() const
{ return rect.getGlobalBounds(); }

void TextBox::setFocused(bool f)
{ is_focused = f; rect.setOutlineColor(f ? Color::Cyan : Color(100, 100, 100)); if (f)
cursorBlinkClock.restart(); }

bool TextBox::isFocused() const noexcept
{ return is_focused; }

bool TextBox::checkClick(Vector2f m)
{ return rect.getGlobalBounds().contains(m); }

void TextBox::setOnChangeCallback(function<void()> cb)
{ onTextChange = cb; }

void TextBox::setEditable(bool editable)
{ rect.setFillColor(editable ? Color(30, 30, 30) : Color(20, 20, 25)); is_editable =
editable; }

void TextBox::handleMouseWheel(float delta, bool shiftPressed)
{
        if (!is_focused) return;
        if (shiftPressed) scrollX -= delta * 30.f;
        else scrollY -= delta * 30.f;
        clampScroll();
    }

void TextBox::setText(const wstring& str)
{
        inputString = String(str);
        cursorPos = selectAnchor = inputString.getSize();
        updatePreferredColumn = true;
        updateText();
    }

string TextBox::getText() const
```

```
{
    auto utf8 = inputString.toUtf8();
    return string(utf8.begin(), utf8.end());
  }

void TextBox::handleTextEntered(char32_t unicode)
{
    if (!is_focused || !is_editable) return;
    updatePreferredColumn = true;
    if (unicode == 8) {
      bool deleted = deleteSelection();
      if (!deleted && cursorPos > 0) {
        inputString.erase(cursorPos - 1, 1);
        cursorPos--; selectAnchor = cursorPos; updateText(); deleted = true;
      }
      if (deleted && onTextChange) onTextChange();
    }
    else if (unicode == 13 || unicode == 10) {
      size_t lineStart = cursorPos;
      while (lineStart > 0 && inputString[lineStart - 1] != '\n') { lineStart--; }
      size_t indentCount = 0;
      while (lineStart + indentCount < inputString.getSize() &&
(inputString[lineStart + indentCount] == ' ' || inputString[lineStart + indentCount]
== '\t')) {
        indentCount++;
      }
      String baseIndent = inputString.substring(lineStart, indentCount);
      bool isBetweenBraces = (cursorPos > 0 && inputString[cursorPos - 1] ==
'{' && cursorPos < inputString.getSize() && inputString[cursorPos] == '}');
      bool isAfterOpeningBrace = (cursorPos > 0 && inputString[cursorPos - 1]
== '{' && !isBetweenBraces);
      if (isBetweenBraces) {
        deleteSelection();
        size_t origPos = cursorPos;
        String textToInsert = "\n" + baseIndent + "    \n" + baseIndent;
        inputString.insert(origPos, textToInsert);
        cursorPos = origPos + 1 + baseIndent.getSize() + 4;
        selectAnchor = cursorPos; updateText();
      }
      else if (isAfterOpeningBrace) {
        deleteSelection();
        size_t origPos = cursorPos;
        String textToInsert = "\n" + baseIndent + "    ";
        inputString.insert(origPos, textToInsert);
        cursorPos = origPos + textToInsert.getSize();
        selectAnchor = cursorPos; updateText();
      }
      else {
        deleteSelection();
        size_t origPos = cursorPos;
        String textToInsert = "\n" + baseIndent;
        inputString.insert(origPos, textToInsert);
        cursorPos = origPos + textToInsert.getSize();
        selectAnchor = cursorPos; updateText();
      }
      if (onTextChange) onTextChange();
    }
    else if (unicode >= 32) {
      insertText(unicode);
      if (onTextChange) onTextChange();
    }
  }

void TextBox::handleKeyPressed(Keyboard::Key key, bool ctrl, bool shift)
{
    if (!is_focused) return;
    if (ctrl && key == Keyboard::Key::C) {
      if (cursorPos != selectAnchor) {
        size_t s = min(cursorPos, selectAnchor), e = max(cursorPos,
selectAnchor);
        Clipboard::setString(inputString.substring(s, e - s));
      }
    }
    if (!is_editable) return;
    if (ctrl && key == Keyboard::Key::V) {
      insertText(Clipboard::getString());
      updatePreferredColumn = true;
      if (onTextChange) onTextChange();
    }
```

```
        else if (ctrl && key == Keyboard::Key::X) {
            if (cursorPos != selectAnchor) {
                size_t s = min(cursorPos, selectAnchor), e = max(cursorPos,
selectAnchor);
                Clipboard::setString(inputString.substring(s, e - s));
                deleteSelection();
                updatePreferredColumn = true;
                if (onTextChange) onTextChange();
            }
        }
        else if (key == Keyboard::Key::Left) {
            if (cursorPos > 0) cursorPos--;
            if (!shift) selectAnchor = cursorPos;
            updatePreferredColumn = true; updateText();
        }
        else if (key == Keyboard::Key::Right) {
            if (cursorPos < inputString.getSize()) cursorPos++;
            if (!shift) selectAnchor = cursorPos;
            updatePreferredColumn = true; updateText();
        }
        else if (key == Keyboard::Key::Up || key == Keyboard::Key::Down) {
            vector<size_t> lineStarts; lineStarts.push_back(0);
            for (size_t i = 0; i < inputString.getSize(); ++i) {
                if (inputString[i] == '\n') lineStarts.push_back(i + 1);
            }
            size_t curLineIdx = 0;
            for (size_t i = 0; i < lineStarts.size(); ++i) {
                if (cursorPos >= lineStarts[i]) curLineIdx = i; else break;
            }
            size_t curColumn = cursorPos - lineStarts[curLineIdx];
            if (updatePreferredColumn) { preferredColumn = curColumn;
updatePreferredColumn = false; }
            if (key == Keyboard::Key::Up) {
                if (curLineIdx > 0) {
                    size_t targetLineIdx = curLineIdx - 1;
                    size_t targetLineStart = lineStarts[targetLineIdx];
                    size_t targetLineEnd = lineStarts[curLineIdx] - 1;
                    size_t targetLineLen = targetLineEnd - targetLineStart;
                    size_t targetColumn = min(preferredColumn, targetLineLen);
                    cursorPos = targetLineStart + targetColumn;
                }
            }
            else {
                if (curLineIdx + 1 < lineStarts.size()) {
                    size_t targetLineIdx = curLineIdx + 1;
                    size_t targetLineStart = lineStarts[targetLineIdx];
                    size_t targetLineEnd = (targetLineIdx + 1 < lineStarts.size()) ?
lineStarts[targetLineIdx + 1] : inputString.getSize();
                    size_t targetLineLen = targetLineEnd - targetLineStart;
                    size_t targetColumn = min(preferredColumn, targetLineLen);
                    cursorPos = targetLineStart + targetColumn;
                }
            }
            if (!shift) selectAnchor = cursorPos; updateText();
        }
        else if (key == Keyboard::Key::Delete) {
            bool deleted = deleteSelection();
            if (!deleted && cursorPos < inputString.getSize()) {
                inputString.erase(cursorPos, 1); updateText(); deleted = true;
            }
            updatePreferredColumn = true;
            if (deleted && onTextChange) onTextChange();
        }
    }

void TextBox::handleMousePress(Vector2f pos)
{
        if (!is_focused) return;
        if (getVScrollRect().contains(pos)) { isDraggingVScroll = true; return; }
        if (getHScrollRect().contains(pos)) { isDraggingHScroll = true; return; }
        cursorPos = selectAnchor = getIndexAtPos(pos);
        isDragging = true; updatePreferredColumn = true; updateText();
    }

void TextBox::handleMouseMove(Vector2f pos)
{
        if (!is_focused) return;
        if (isDraggingVScroll) {
            float contentHeight = sfText.getLocalBounds().size.y + 50.f;
```

```
            float ratio = canvas.getSize().y / contentHeight;
            float barHeight = max(20.f, canvas.getSize().y * ratio);
            float trackHeight = canvas.getSize().y - barHeight;
            if (trackHeight > 0) {
                float relativeY = pos.y - rect.getPosition().y - barHeight / 2.f;
                float newRatio = clamp(relativeY / trackHeight, 0.f, 1.f);
                scrollY = newRatio * (contentHeight - canvas.getSize().y);
            }
            return;
        }
        if (isDraggingHScroll) {
            float contentWidth = sfText.getLocalBounds().size.x + 20.f;
            float ratio = canvas.getSize().x / contentWidth;
            float barWidth = max(20.f, canvas.getSize().x * ratio);
            float trackWidth = canvas.getSize().x - barWidth;
            if (trackWidth > 0) {
                float relativeX = pos.x - rect.getPosition().x - barWidth / 2.f;
                float newRatio = clamp(relativeX / trackWidth, 0.f, 1.f);
                scrollX = newRatio * (contentWidth - canvas.getSize().x);
            }
            return;
        }
        if (isDragging) { cursorPos = getIndexAtPos(pos); updatePreferredColumn =
true; updateText(); }
    }

void TextBox::handleMouseRelease()
{ isDragging = false; isDraggingVScroll = false; isDraggingHScroll = false; }

void TextBox::draw(RenderWindow& w)
{
        w.draw(rect);
        canvas.clear(is_editable ? Color(30, 30, 30) : Color(20, 20, 25));
        sfText.setPosition({ 10.f - scrollX, 10.f - scrollY });
        if (cursorPos != selectAnchor) {
            size_t s = min(cursorPos, selectAnchor), e = max(cursorPos, selectAnchor);
            for (size_t i = s; i < e; ++i) {
                if (inputString[i] == '\n' || inputString[i] == '\r') continue;
                Vector2f p1 = sfText.findCharacterPos(i), p2 = sfText.findCharacterPos(i
+ 1);
                float width = p2.x - p1.x; if (width <= 0) width = 8.f;
                RectangleShape hl({ width, static_cast<float>(sfText.getCharacterSize()
+ 4) });
                hl.setPosition({ p1.x, p1.y + 2.f }); hl.setFillColor(Color(0, 100, 200,
150));
                canvas.draw(hl);
            }
        }
        canvas.draw(sfText);
        if (is_focused && is_editable) {
            if (cursorBlinkClock.getElapsedTime().asMilliseconds() > 500) {
cursorVisible = !cursorVisible; cursorBlinkClock.restart(); }
            if (cursorVisible) {
                Vector2f cPos = sfText.findCharacterPos(cursorPos);
                RectangleShape cursorRect({ 2.f,
static_cast<float>(sfText.getCharacterSize()) });
                cursorRect.setPosition({ cPos.x, cPos.y + 4.f });
cursorRect.setFillColor(Color::White);
                canvas.draw(cursorRect);
            }
        }
        canvas.display();
        w.draw(canvasSprite);
        FloatRect vRect = getVScrollRect();
        if (vRect.size.y > 0) {
            RectangleShape vScroll({ 6.f, vRect.size.y });
            vScroll.setFillColor(Color(100, 100, 100, 200));
            vScroll.setPosition({ rect.getPosition().x + canvas.getSize().x - 6.f,
vRect.position.y });
            w.draw(vScroll);
        }
        FloatRect hRect = getHScrollRect();
        if (hRect.size.x > 0) {
            RectangleShape hScroll({ hRect.size.x, 6.f });
            hScroll.setFillColor(Color(100, 100, 100, 200));
            hScroll.setPosition({ hRect.position.x, rect.getPosition().y +
canvas.getSize().y - 6.f });
            w.draw(hScroll);
        }
```

```
}
```

TheoryCarousel.cpp:

```
#include "TheoryCarousel.hpp"

TheoryCarousel::TheoryCarousel(const ResourceManager& rm)
{
    for (int i = 0; i < totalSlides; ++i) {
        RectangleShape box({ 1200.f, 750.f });
        box.setOrigin({ 600.f, 375.f });
        box.setFillColor(Color::White);
        box.setTexture(&rm.theoryTextures[i]);
        box.setOutlineThickness(6.f);
        box.setOutlineColor(Color::White);
        slides.push_back(box);
    }
}

void TheoryCarousel::reset()
{ currentIdx = 0; currentOffset = 0.f; targetOffset = 0.f; }

int TheoryCarousel::getCurrentIdx() const noexcept
{ return currentIdx; }

int TheoryCarousel::getTotalSlides() const noexcept
{ return totalSlides; }

void TheoryCarousel::moveLeft()
{ if (currentIdx > 0) { currentIdx--; targetOffset += 1400.f; } }

void TheoryCarousel::moveRight()
{ if (currentIdx < totalSlides - 1) { currentIdx++; targetOffset -= 1400.f; } }

void TheoryCarousel::update(float dt)
{
    currentOffset += (targetOffset - currentOffset) * 10.f * dt;
    for (int i = 0; i < totalSlides; ++i) {
        float xPos = 960.f + (i * 1400.f) + currentOffset;
        slides[i].setPosition({ xPos, 500.f });
    }
}

void TheoryCarousel::draw(RenderWindow& w)
{ for (auto& s : slides) w.draw(s); }
```

TheoryCarousel.hpp:

```
#pragma once

#include "Common.hpp"
#include "ResourceManager.hpp"

class TheoryCarousel {

private:

    vector<RectangleShape> slides;
    int currentIdx = 0;
    int totalSlides = 11;
    float currentOffset = 0.f;
    float targetOffset = 0.f;
public:
    TheoryCarousel(const ResourceManager& rm);
    void reset();
    int getCurrentIdx() const noexcept;
    int getTotalSlides() const noexcept;
    void moveLeft();
    void moveRight();
    void update(float dt);
    void draw(RenderWindow& w);
};
```

Utils.cpp:

```
#include "Utils.hpp"

bool isOutOfBounds(Vector2f point) {
    return (point.x < 10.0f || point.x > 1910.0f || point.y < 10.0f || point.y > 1000.0f);
}
bool isPathOutOfBounds(Vector2f start, Vector2f end) { return isOutOfBounds(start) || isOutOfBounds(end) || isOutOfBounds(Vector2f(end.x, start.y)); }
bool isSegmentOverlapping(Vector2f s, Vector2f e, const vector<WireConnection>& connections, const vector<Wire>& oneWire) {
    auto checkOverlap = [&](Vector2f a, Vector2f b, Vector2f c, Vector2f d) {
        if (abs(a.y - b.y) < 1.f && abs(c.y - d.y) < 1.f && abs(a.y - c.y) < 1.f) {
            float minX1 = min(a.x, b.x), maxX1 = max(a.x, b.x); float minX2 = min(c.x, d.x), maxX2 = max(c.x, d.x);
            return max(minX1, minX2) < min(maxX1, maxX2);
        }
        if (abs(a.x - b.x) < 1.f && abs(c.x - d.x) < 1.f && abs(a.x - c.x) < 1.f) {
            float minY1 = min(a.y, b.y), maxY1 = max(a.y, b.y); float minY2 = min(c.y, d.y), maxY2 = max(c.y, d.y);
            return max(minY1, minY2) < min(maxY1, maxY2);
        }
        return false;
        };
    for (const auto& conn : connections) {
        for (const auto& wire : conn.segments) {
            if (checkOverlap(s, e, wire.getStart(), wire.getEnd())) return true;
        }
    }
    for (const auto& wire : oneWire) { if (checkOverlap(s, e, wire.getStart(), wire.getEnd())) return true; }
    return false;
}
```

Utils.hpp:

```
#pragma once

#include "Common.hpp"
#include "Globals.hpp"
#include "Pin.hpp"
#include "ResourceManager.hpp"
#include "Wire.hpp"

bool isOutOfBounds(Vector2f point);
bool isPathOutOfBounds(Vector2f start, Vector2f end);
bool isSegmentOverlapping(Vector2f s, Vector2f e, const vector<WireConnection>& connections, const vector<Wire>& oneWire);
void add_pins(vector<Pin>& pins, const ResourceManager& rm, const Sprite& board);
string getPinLabel(int pinNum);
void generateLevelTask();
```

```
void add_pins(vector<Pin>& pins, const ResourceManager& rm, const Sprite& board) {
   FloatRect bounds = board.getGlobalBounds();
   float pinSpacing = 17.5f;
   float topY = 25.f;
   float bottomY = 252.5f;
   int leftPins[] = { 19, 18, -6, -2, 13, 12, 11, 10, 9, 8 };
   for (int i = 0; i < 10; ++i) {
      pins.emplace_back(Vector2f{ bounds.position.x + 142.5f + (i * pinSpacing), bounds.position.y + topY }, rm);
      pins.back().setArduinoPin(leftPins[i]);
      if (leftPins[i] == -2) pins.back().setRole(PinRole::SourceGND);
   }
   int rightPins[] = { 7, 6, 5, 4, 3, 2, 1, 0 };
   for (int i = 0; i < 8; ++i) {
      pins.emplace_back(Vector2f{ bounds.position.x + 322.5f + (i * pinSpacing), bounds.position.y + topY }, rm);
      pins.back().setArduinoPin(rightPins[i]);
   }
   pinSpacing = 16.f;
   int powerPins[] = { -8, -7, -4, -3, -5, -2, -2, -1 };
   for (int i = 0; i < 8; ++i) {
      pins.emplace_back(Vector2f{ bounds.position.x + 200.f + (i * pinSpacing + 4.f), bounds.position.y + bottomY }, rm);
      pins.back().setArduinoPin(powerPins[i]);
      if (powerPins[i] == -3) pins.back().setRole(PinRole::Source5V);
      if (powerPins[i] == -4) pins.back().setRole(PinRole::Source3_3V);
      if (powerPins[i] == -2) pins.back().setRole(PinRole::SourceGND);
   }
   pinSpacing = 18.5f;
   for (int i = 0; i < 6; ++i) {
      pins.emplace_back(Vector2f{ bounds.position.x + 350.f + (i * pinSpacing), bounds.position.y + bottomY }, rm);
      pins.back().setArduinoPin(14 + i);
   }
}

string getPinLabel(int pinNum) {
   if (pinNum >= 14 && pinNum <= 19) return "A" + to_string(pinNum - 14);
return to_string(pinNum);
}

void generateLevelTask() {
   int availablePins[] = { 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19 };
   int idx1 = rand() % 18; int idx2 = rand() % 17; if (idx2 >= idx1) idx2++;
   genPinHigh = availablePins[idx1]; genPinLow = availablePins[idx2]; genServoPin = 2 + (rand() % 12);
   btnAsSwitchLevel3 = (rand() % 2 == 0); btnAsSwitchLevel4 = (rand() % 2 == 0);
}
```

VirtualArduino.cpp:

```
#include "VirtualArduino.hpp"

   VirtualArduino::VirtualArduino(function<vector<Pin*>()> getPins, CircuitManager& cm, ComponentNetwork& net)
      : getAllPinsFunc(getPins), circuitManager(cm), networkSimulator(net)
{
   }


void VirtualArduino::pinMode(int pinNum, int mode)
{
      lock_guard<mutex> lock(boardMutex);
      if (mode == 1) {
         outputPins.insert(pinNum);
         auto allPins = getAllPinsFunc();
         for (Pin* pin : allPins) {
            if (pin->getArduinoPin() == pinNum && pin->getRole() == PinRole::Normal) pin->setRole(PinRole::SourceGND);
         }
         networkSimulator.resolveLogic(allPins, circuitManager);
      }
      else {
         outputPins.erase(pinNum);
         auto allPins = getAllPinsFunc();
         for (Pin* pin : allPins) {
            if (pin->getArduinoPin() == pinNum) pin->setRole(PinRole::Normal);
         }
```

VirtualArduino.hpp:

```
#pragma once

#include "Common.hpp"
#include "Globals.hpp"
#include "Pin.hpp"
#include "CircuitManager.hpp"
#include "ComponentNetwork.hpp"

class VirtualArduino {

private:

   function<vector<Pin*>()> getAllPinsFunc;
   CircuitManager& circuitManager;
   ComponentNetwork& networkSimulator;
   unordered_set<int> outputPins;
public:
   VirtualArduino(function<vector<Pin*>()> getPins, CircuitManager& cm, ComponentNetwork& net);
   void pinMode(int pinNum, int mode);
   void digitalWrite(int pinNum, int value);
   int digitalRead(int pinNum);
   void resetAllPins();
};
```

```
            networkSimulator.resolveLogic(allPins, circuitManager);
        }
    }

void VirtualArduino::digitalWrite(int pinNum, int value)
{
        if (pinNum < 0) return;
        lock_guard<mutex> lock(boardMutex);
        if (outputPins.find(pinNum) == outputPins.end()) return;
        auto allPins = getAllPinsFunc();
        bool found = false;
        for (Pin* pin : allPins) {
            if (pin->getArduinoPin() == pinNum) {
                pin->setRole(value == 1 ? PinRole::Source5V : PinRole::SourceGND);
                found = true;
            }
        }
        if (found) networkSimulator.resolveLogic(allPins, circuitManager);
    }

int VirtualArduino::digitalRead(int pinNum)
{
        lock_guard<mutex> lock(boardMutex);
        auto allPins = getAllPinsFunc();
        for (Pin* pin : allPins) {
            if (pin->getArduinoPin() == pinNum) return pin->getState() ==
LogicLevel::High ? 1 : 0;
        }
        return 0;
    }

void VirtualArduino::resetAllPins()
{
        lock_guard<mutex> lock(boardMutex);
        outputPins.clear();
        auto allPins = getAllPinsFunc();
        for (Pin* pin : allPins) {
            if (pin->getArduinoPin() >= 0) pin->setRole(PinRole::Normal);
            else if (pin->getArduinoPin() == -2) pin->setRole(PinRole::SourceGND);
            else if (pin->getArduinoPin() == -3) pin->setRole(PinRole::Source5V);
            else if (pin->getArduinoPin() == -4) pin->setRole(PinRole::Source3_3V);
        }
        networkSimulator.resolveLogic(allPins, circuitManager);
    }
```

Wire.cpp:

```
#include "Wire.hpp"

    Wire::Wire(Vector2f s, Vector2f e, Color color)
{ line.setFillColor(color); update(s, e); }

Vector2f Wire::getStart() const noexcept
{ return startPoint; }

Vector2f Wire::getEnd() const noexcept
{ return endPoint; }

void Wire::setColor(Color c)
{ line.setFillColor(c); }

void Wire::update(Vector2f s, Vector2f e)
{
        startPoint = s; endPoint = snapToGrid(s, e);
        Vector2f d = endPoint - startPoint;
        line.setSize({ hypot(d.x, d.y), 4.f });
        line.setOrigin({ 0.f, 2.f }); line.setPosition(s);
        line.setRotation(degrees((float)search_angle(d)));
    }

int Wire::search_angle(Vector2f d)
{
        int a = (int)(atan2(d.y, d.x) * 180.f / 3.14159f);
        if (a >= 45 && a <= 135) return 90;
        if (a > 135 || a < -135) return 180;
        if (a <= -45 && a >= -135) return -90;
        return 0;
    }

Vector2f Wire::snapToGrid(Vector2f s, Vector2f e)
{
```

Wire.hpp:

```
#pragma once

#include "Common.hpp"

class Wire {

private:

    RectangleShape line; Vector2f startPoint; Vector2f endPoint;
public:
    Wire(Vector2f s, Vector2f e, Color color = Color::Red);
    Vector2f getStart() const noexcept;
    Vector2f getEnd() const noexcept;
    void setColor(Color c);
    void update(Vector2f s, Vector2f e);
    static int search_angle(Vector2f d);
    static Vector2f snapToGrid(Vector2f s, Vector2f e);
    int getAngle() const noexcept;
    void draw(RenderWindow& w) const;
};

struct WireConnection {

    vector<Wire> segments;
    uint32_t pin1Id = 0;
    uint32_t pin2Id = 0;
    Color color = Color::Red;

    void setColor(Color c) noexcept;
    void draw(RenderWindow& w) const;
};
```

```
        Vector2f d = e - s; float len = hypot(d.x, d.y);
        int a = search_angle(d);
        if (a == 90) return { s.x, s.y + len };
        if (abs(a) == 180) return { s.x - len, s.y };
        if (a == -90) return { s.x, s.y - len };
        return { s.x + len, s.y };
    }

int Wire::getAngle() const noexcept
{ return search_angle(endPoint - startPoint); }

void Wire::draw(RenderWindow& w) const
{ w.draw(line); }

    void WireConnection::setColor(Color c) noexcept
{ color = c; for (auto& w : segments) w.setColor(c); }

void WireConnection::draw(RenderWindow& w) const
{ for (const auto& seg : segments) seg.draw(w); }
```